

CoMeT

Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost

F. Nisa Bostancı

I. E. Yüksel A. Olgun K. Kanellopoulos Y. C. Tuğrul

A. G. Yağlıkçı M. Sadrosadati O. Mutlu

https://github.com/CMU-SAFARI/CoMeT

SAFARI

ETH zürich

Executive Summary

Problem: As DRAM becomes more vulnerable to read disturbance, existing RowHammer mitigation techniques either prevent bitflips
(1) at low performance cost but with high area overheads or
(2) at low area cost but with prohibitively large performance and energy overheads

Goal: Prevent RowHammer bitflips with low area, performance, and energy overheads in highly RowHammer-vulnerable DRAM-based systems

Key Idea: Use low-cost and scalable hash-based counters to accurately track DRAM rows

CoMeT:
- tracks most DRAM rows with scalable hash-based counters by employing the Count-Min-Sketch technique to achieve a low area cost
- tracks only a small set of DRAM rows that are activated many times with highly accurate per-DRAM-row activation counters to reduce performance penalties

Evaluation: CoMeT achieves a good trade-off between area, performance and energy costs
- incurs significantly less area overhead (74.2×) compared to the state-of-the-art technique
- outperforms the state-of-the-art technique (by up to 39.1%)

https://github.com/CMU-SAFARI/CoMeT

Outline

Background and Problem

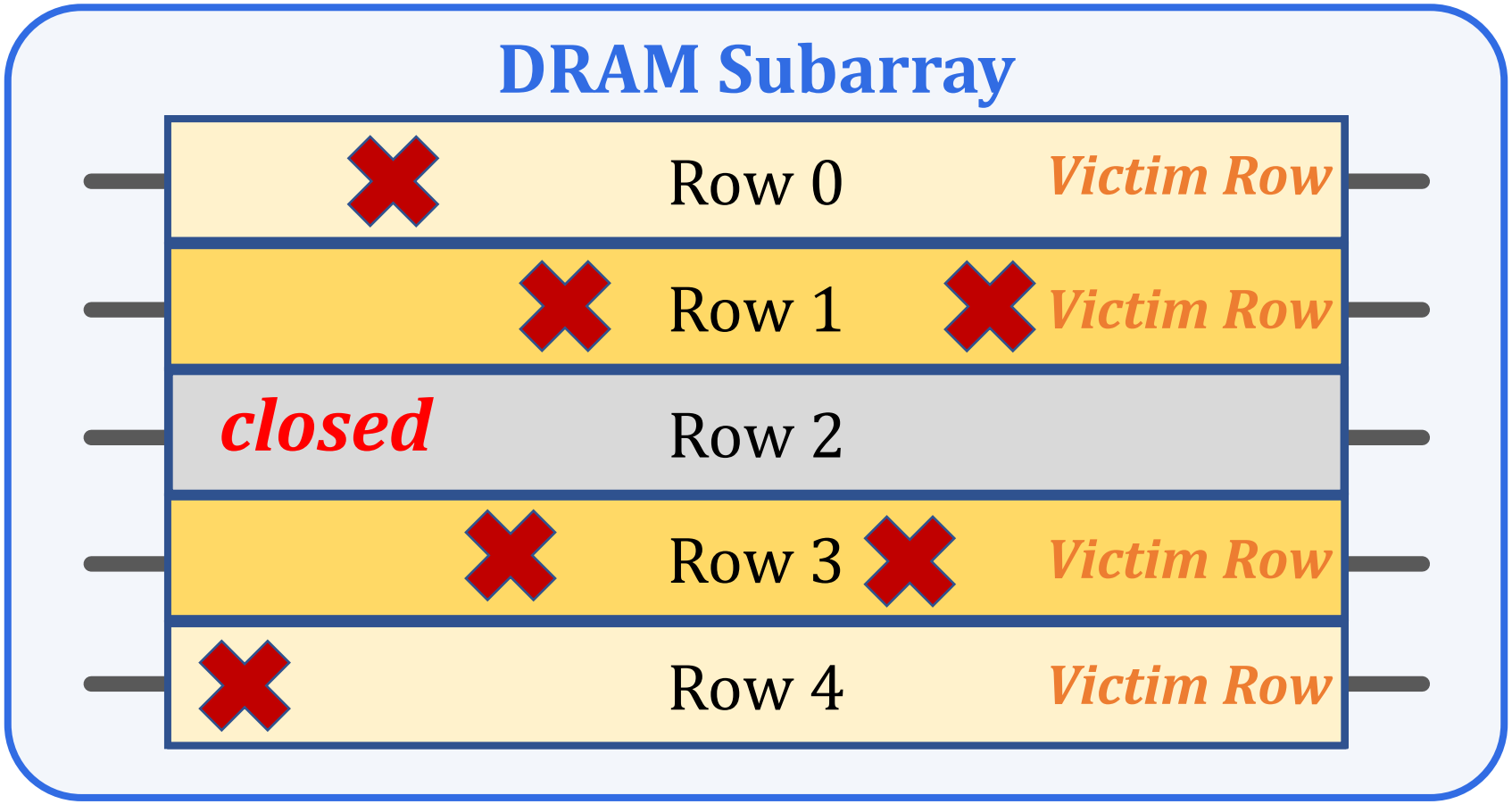
Goal and Key Idea

CoMeT: Count-Min-Sketch-based Row Tracking

Evaluation

Conclusion

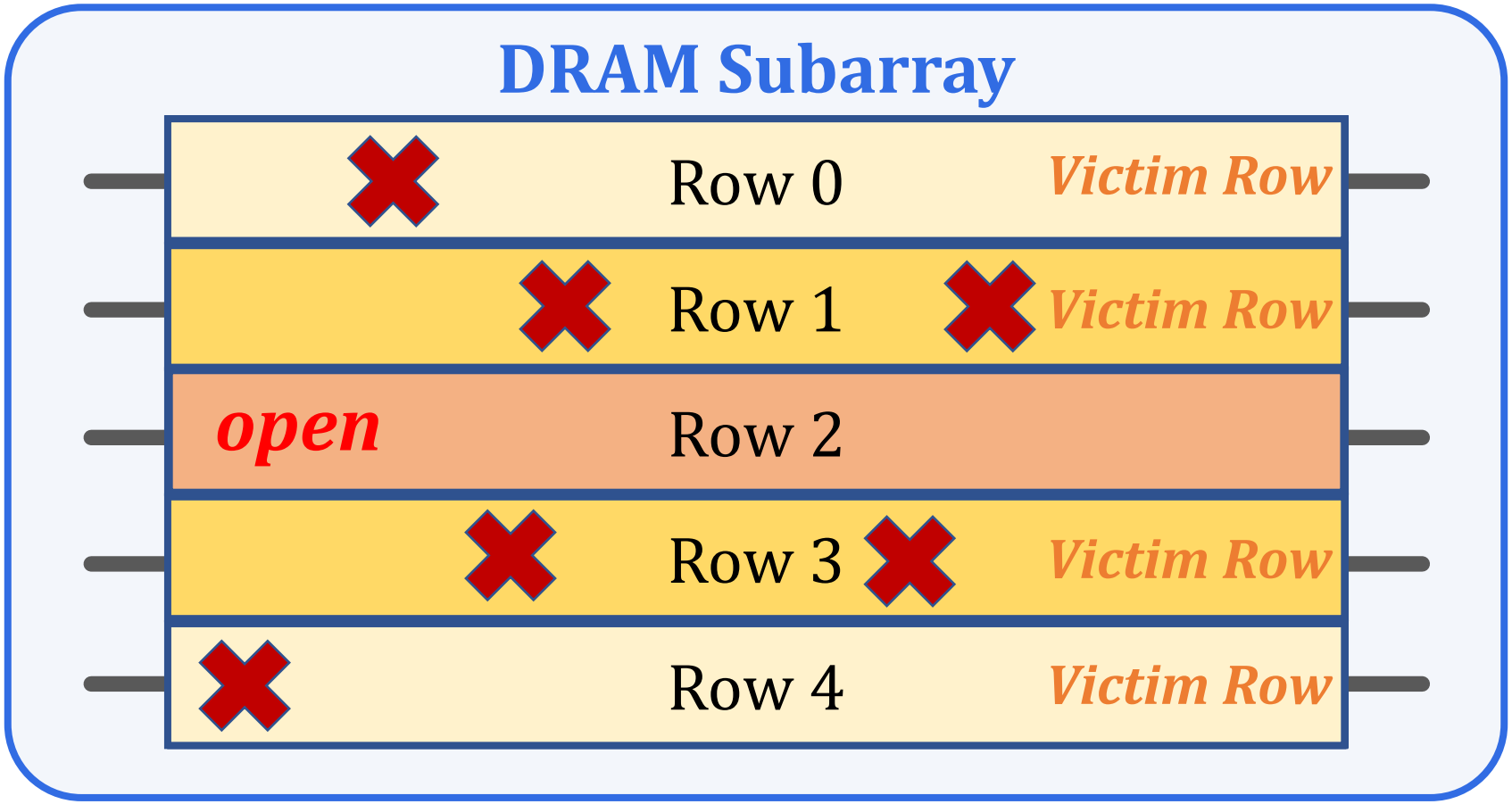
Read Disturbance Vulnerabilities (I)



Repeatedly **opening** (activating) and **closing** (precharging) a DRAM row causes **RowHammer bitflips** in nearby cells

[Kim+ ISCA'20]

Read Disturbance Vulnerabilities (I)



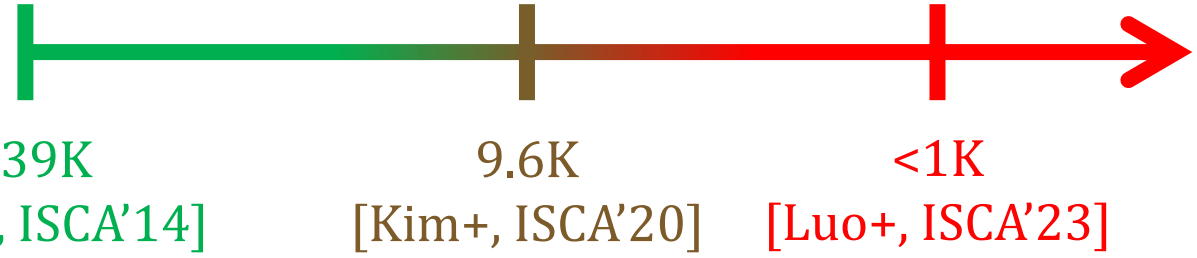
The minimum number of activations that causes a bitflip is called **the RowHammer threshold**

[Kim+ ISCA'20]

Read Disturbance Vulnerabilities (II)

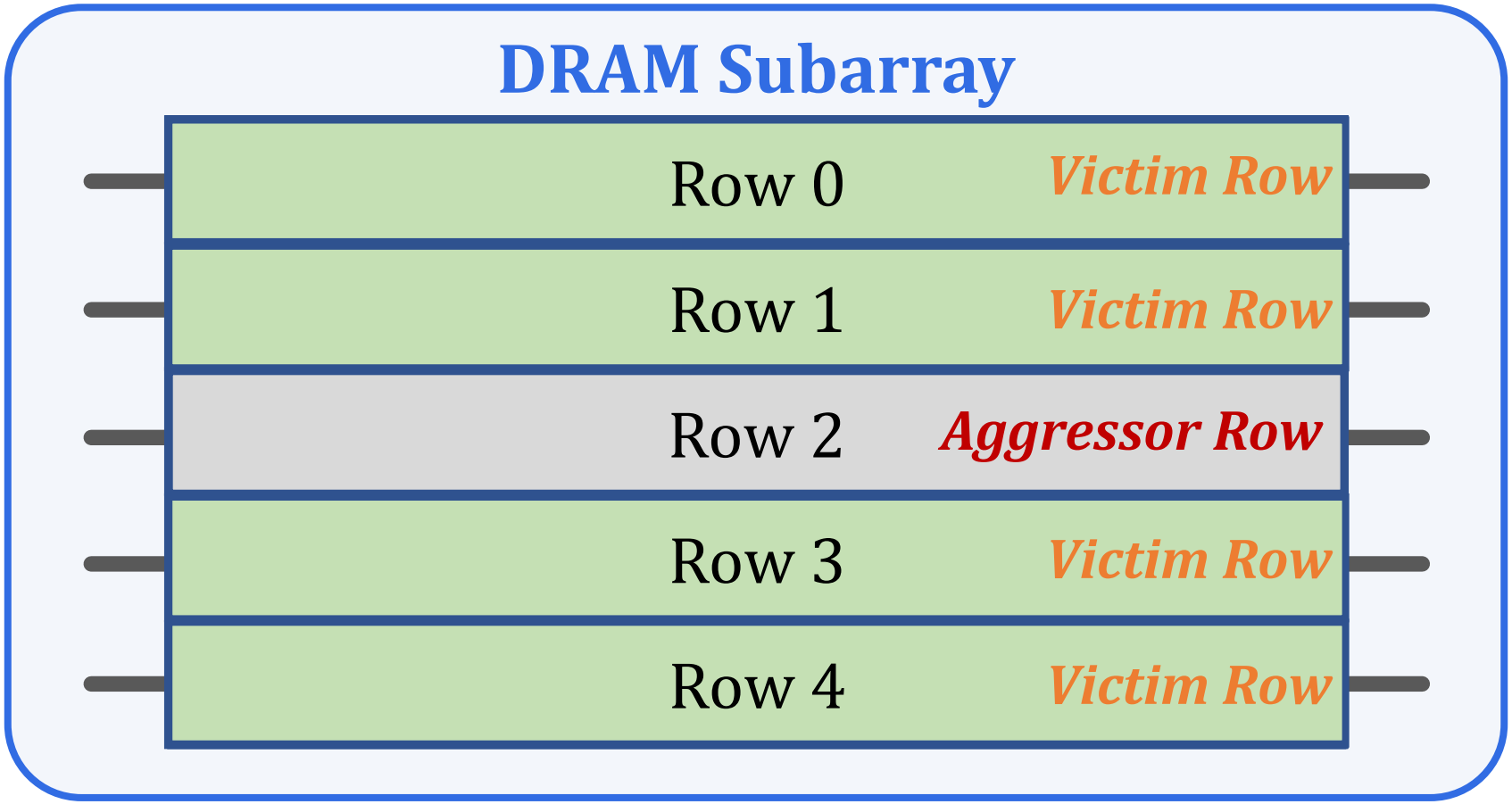
- DRAM chips are more vulnerable to read disturbance today

- Read disturbance bitflips occur at much lower activation counts (more than two orders of magnitude decrease in less than a decade):



Mitigation techniques against read disturbance attacks need to be **effective** and **efficient** for highly vulnerable systems

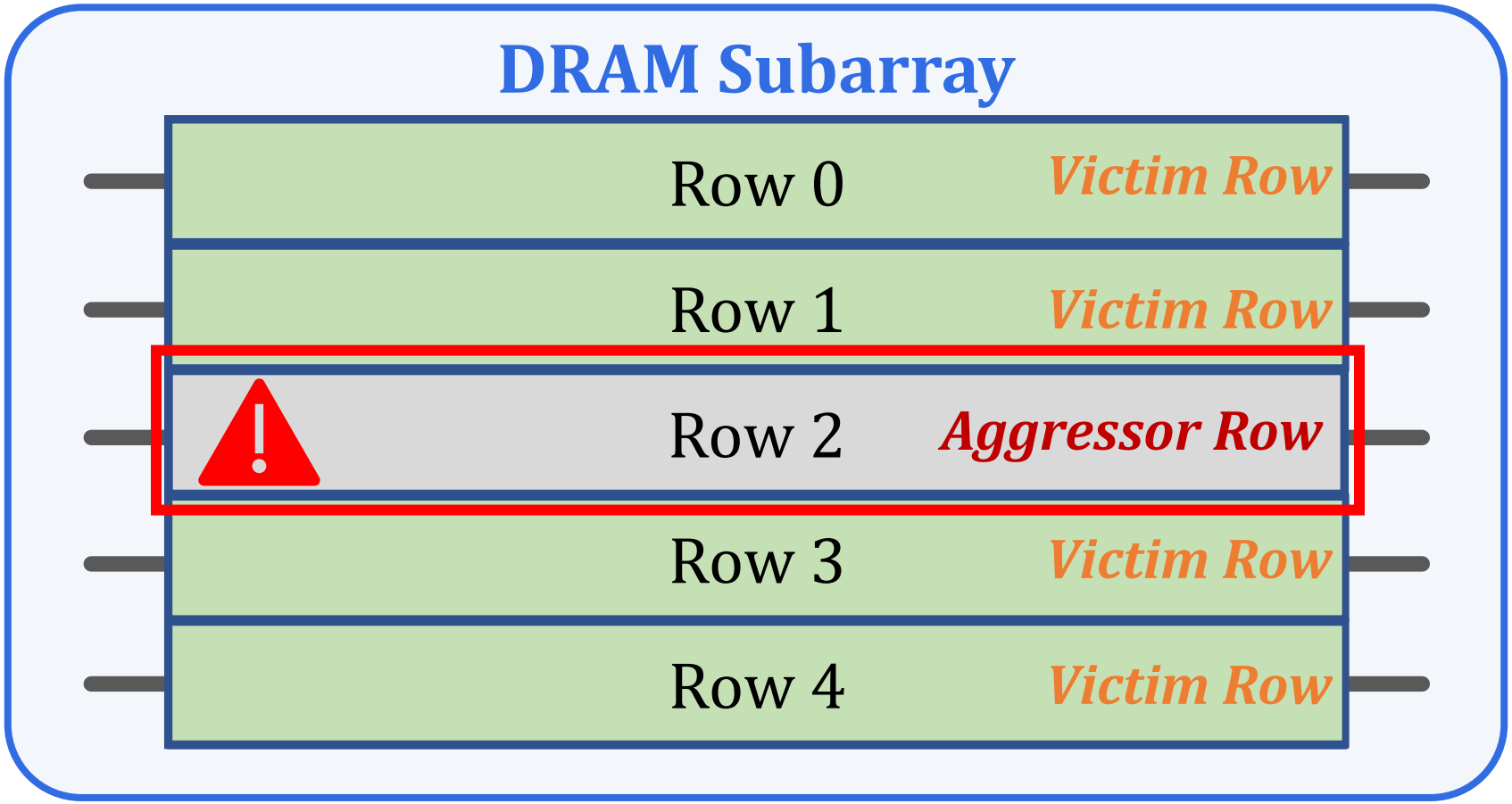
Existing RowHammer Mitigations (I): Preventive Refresh



Refreshing potential victim rows
mitigates read disturbance bitflips

[Kim+ ISCA'20]

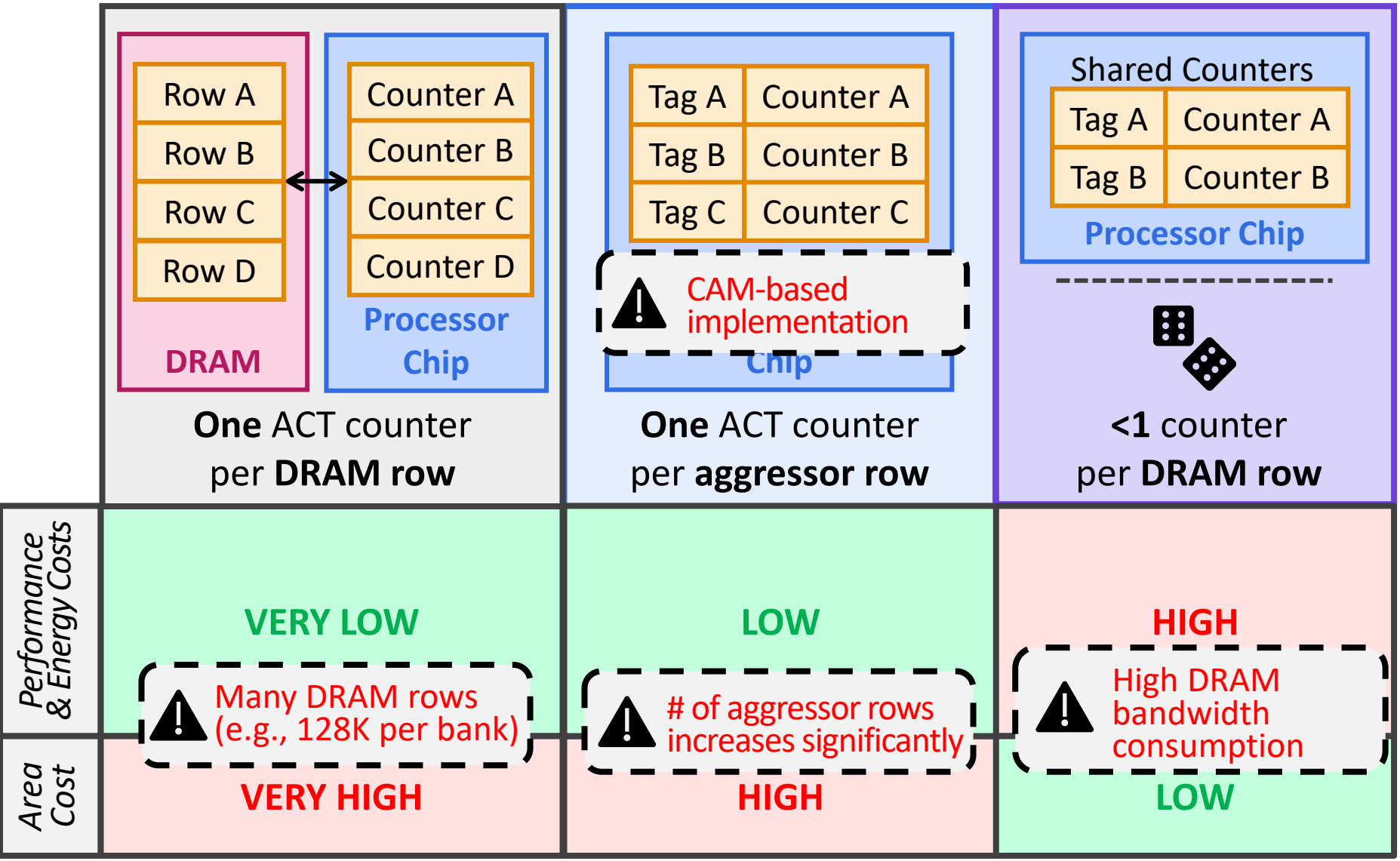
Existing RowHammer Mitigations (II): DRAM Row Activation Tracking



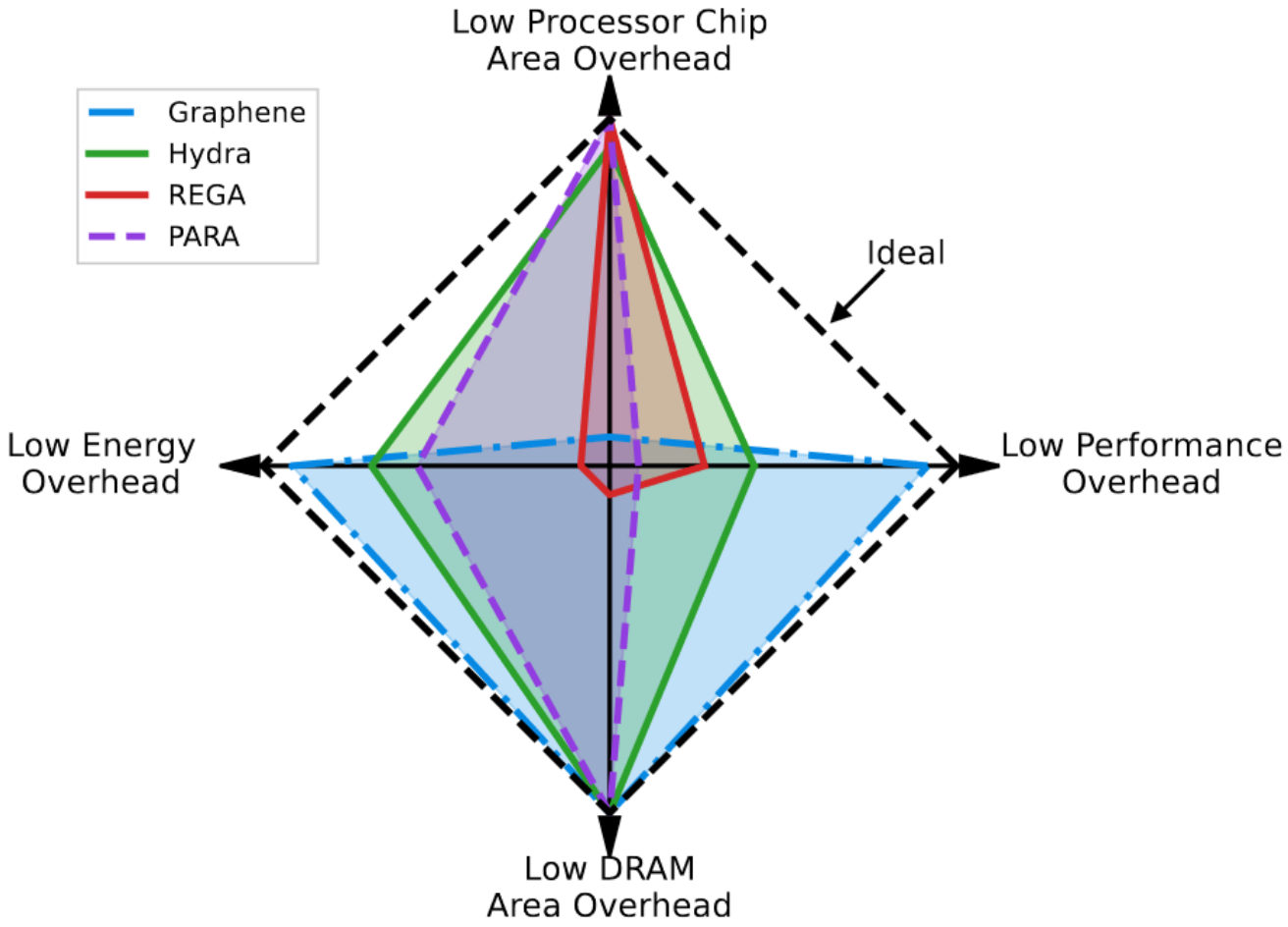
Mitigation techniques track DRAM row activation counts (of aggressor rows) to preventively refresh potential victim rows

[Kim+ ISCA'20]

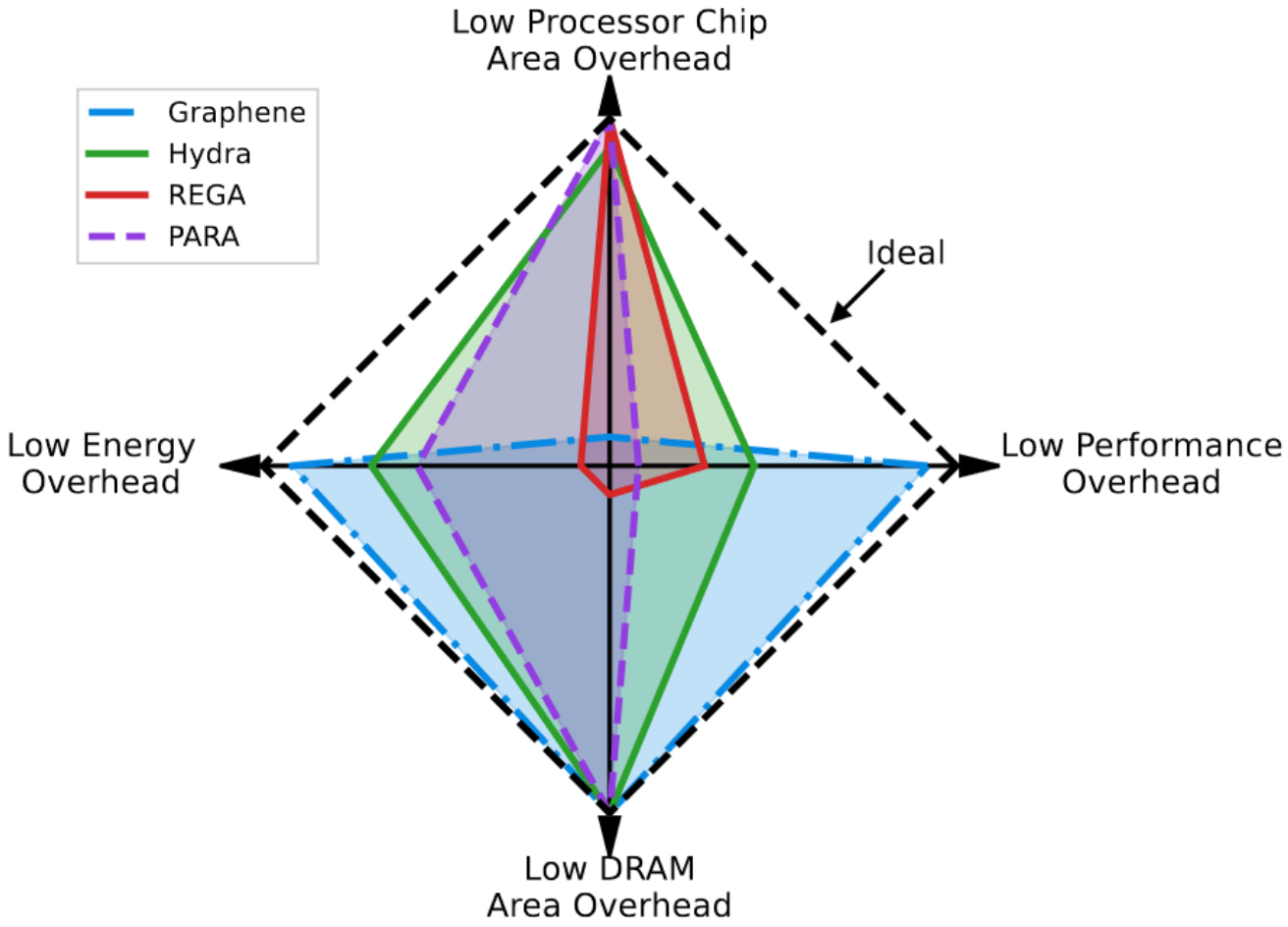
Existing RowHammer Mitigations (III):
Overview of Preventive Refresh-Based Mitigation Techniques



Existing RowHammer Mitigations (IV):
Overhead Trade-Off of State-of-the-Art Mitigation Techniques



Existing RowHammer Mitigations (IV):
Overhead Trade-Off of State-of-the-Art Mitigation Techniques



No existing mitigation technique prevents RowHammer bitflips
at low area, performance and energy costs

Outline

Background and Problem

Goal and Key Idea

CoMeT: Count-Min-Sketch-based Row Tracking

Evaluation

Conclusion

Our Goal

Prevent RowHammer bitflips

with low area, performance, and energy overheads

in highly RowHammer-vulnerable DRAM-based systems

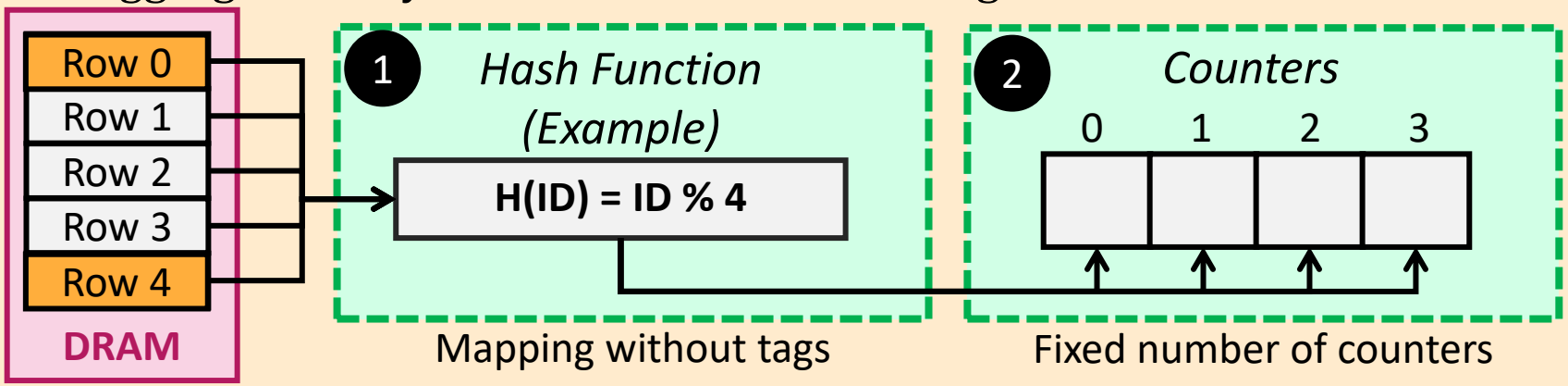
(e.g., a RowHammer threshold of 125)

Key Observation

Hash-based counters are low-cost:

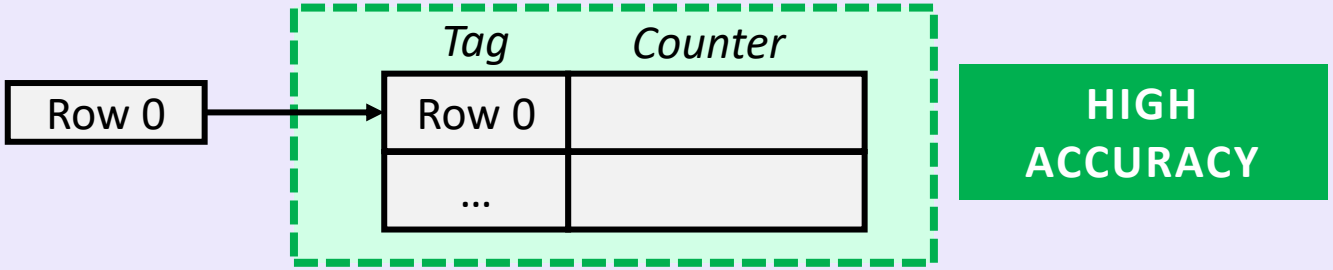
1. can be implemented with low-cost structures and
2. can aggregate many rows' activation counts together

LOW COST



Tag-based counters are highly accurate:

Each one tracks one row's activation count



HIGH ACCURACY

Key Idea

1 Use low-cost and scalable hash-based counters to track most DRAM rows' activations with low area overhead

2 Use highly accurate tag-based counters to track only a small set of DRAM rows to achieve low performance overhead

Outline

Background and Problem

Goal and Key Idea

CoMeT: Count-Min-Sketch-based Row Tracking

Evaluation

Conclusion

CoMeT Overview

Counter Table (CT):

- Maps each DRAM row to a group of low-cost hash-based counters as uniquely as possible by employing the Count-Min Sketch technique
- Triggers a preventive refresh to an aggressor row's victim rows when the aggressor's counter group reaches an activation threshold

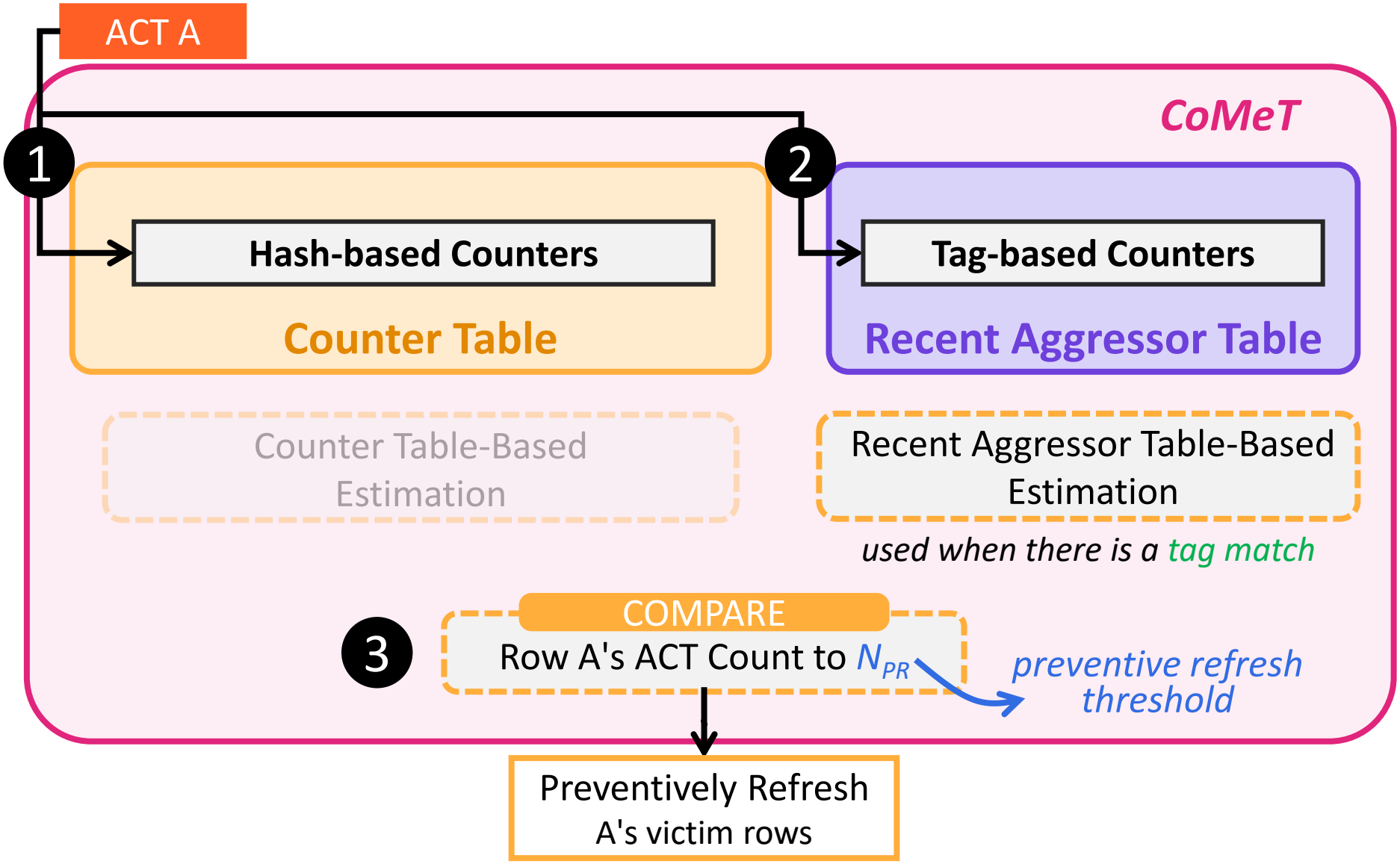
Tracks DRAM row activations at low area cost

Recent Aggressor Table (RAT):

- Allocates highly accurate per-DRAM-row counters for *only* a small set of DRAM rows that are activated many times

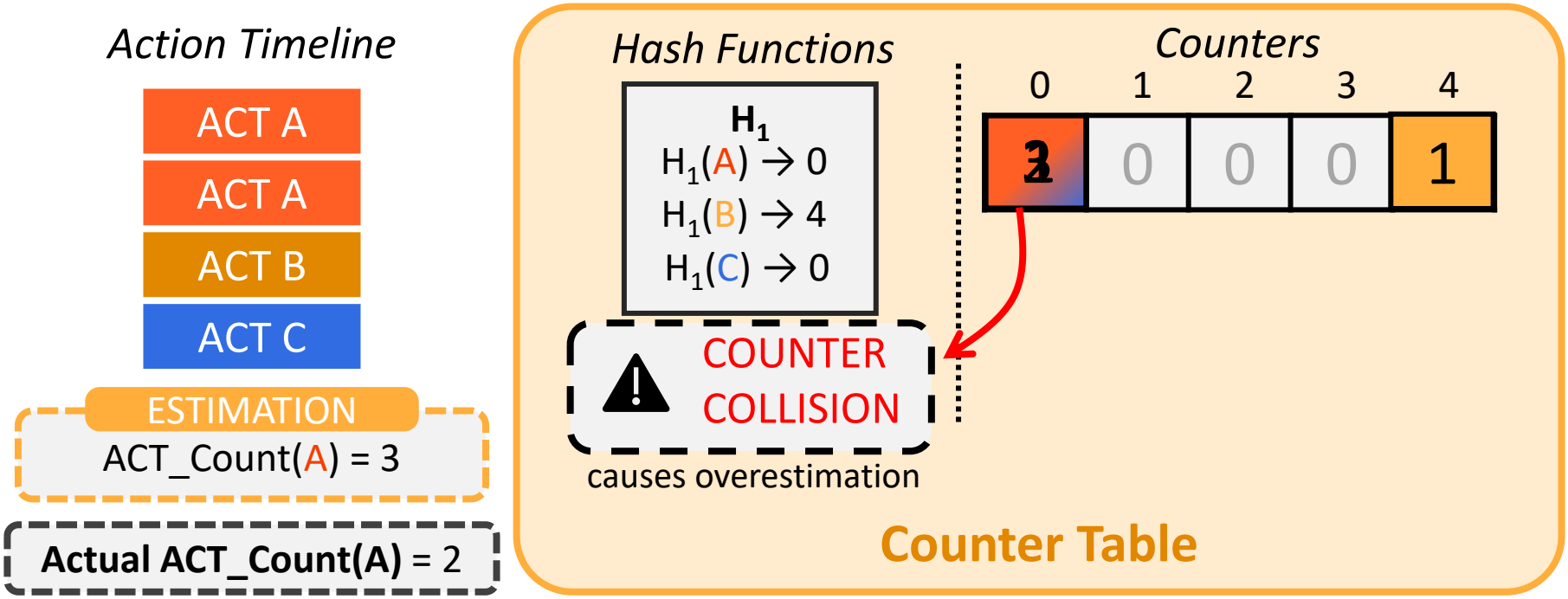
Reduces performance penalties by increasing tracking accuracy

Operation of CoMeT



Counter Table (CT):
Count-Min-Sketch-based Row Tracking

- **Count-Min Sketch:** A hash-based frequent item counting technique



Counter Table (CT):
Count-Min-Sketch-based Row Tracking

- To avoid overestimations, Counter Table implements multiple hash functions

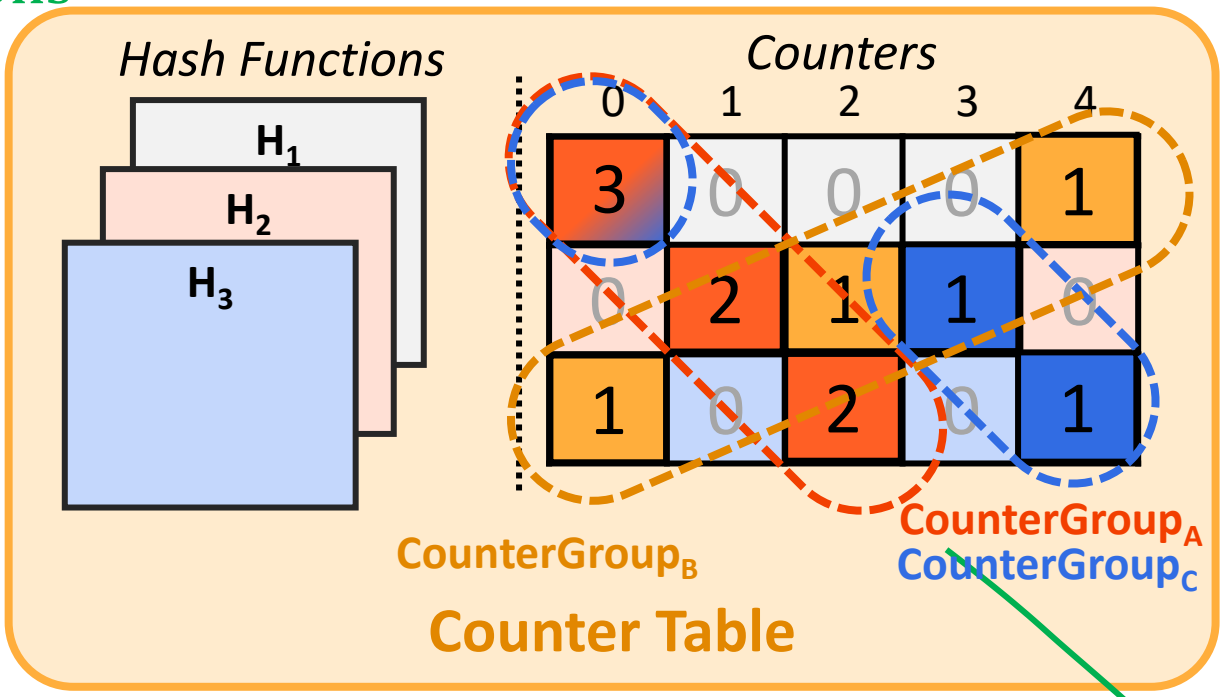
Action Timeline

ACT A
ACT A
ACT B
ACT C

ESTIMATION
ACT_Count(A) = 2

Actual ACT_Count(A) = 2

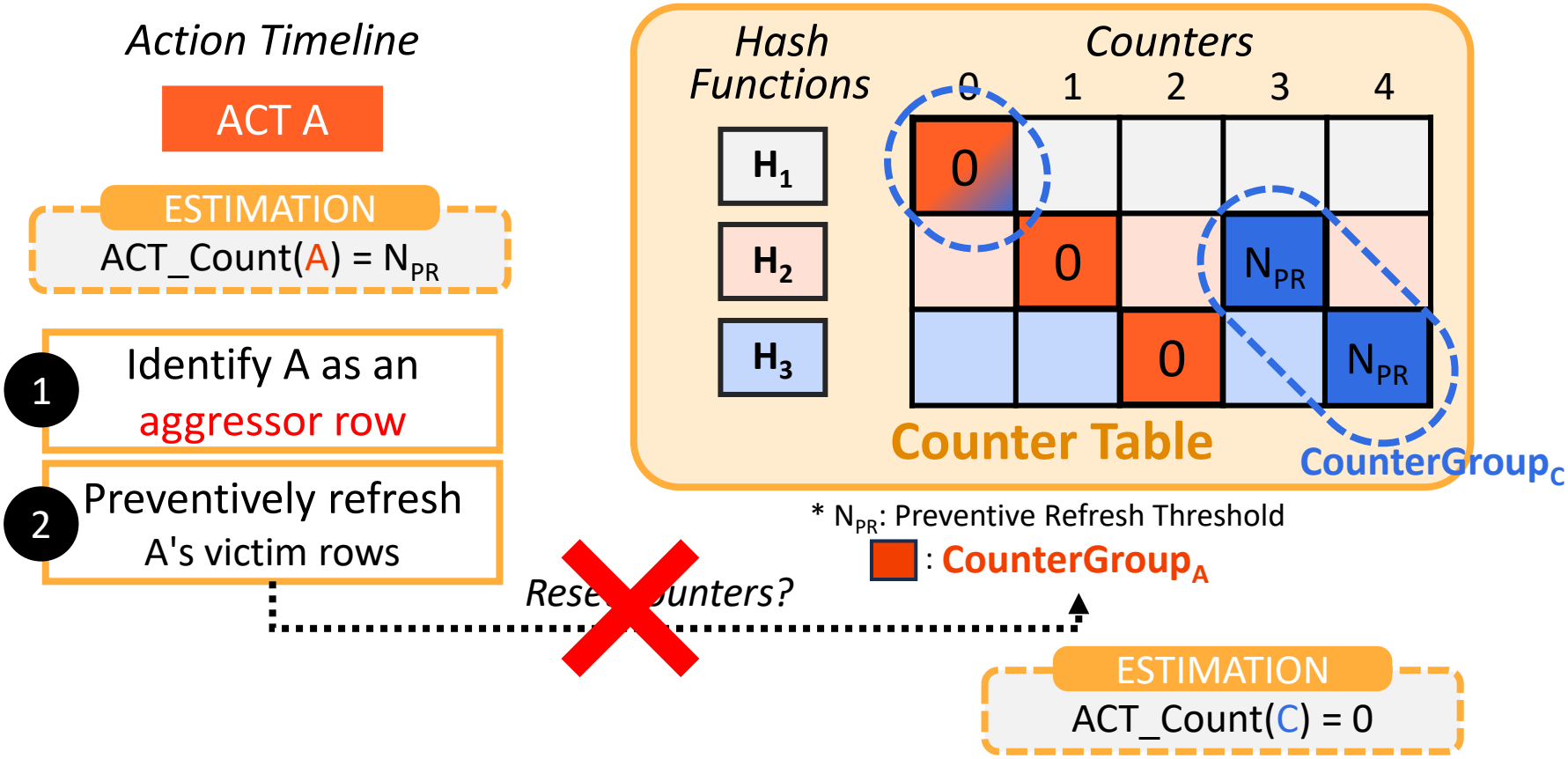
ESTIMATION
ACT_Count(A)
=
MIN($CounterGroup_A$)



The minimum counter value is an upper bound for the actual activation count

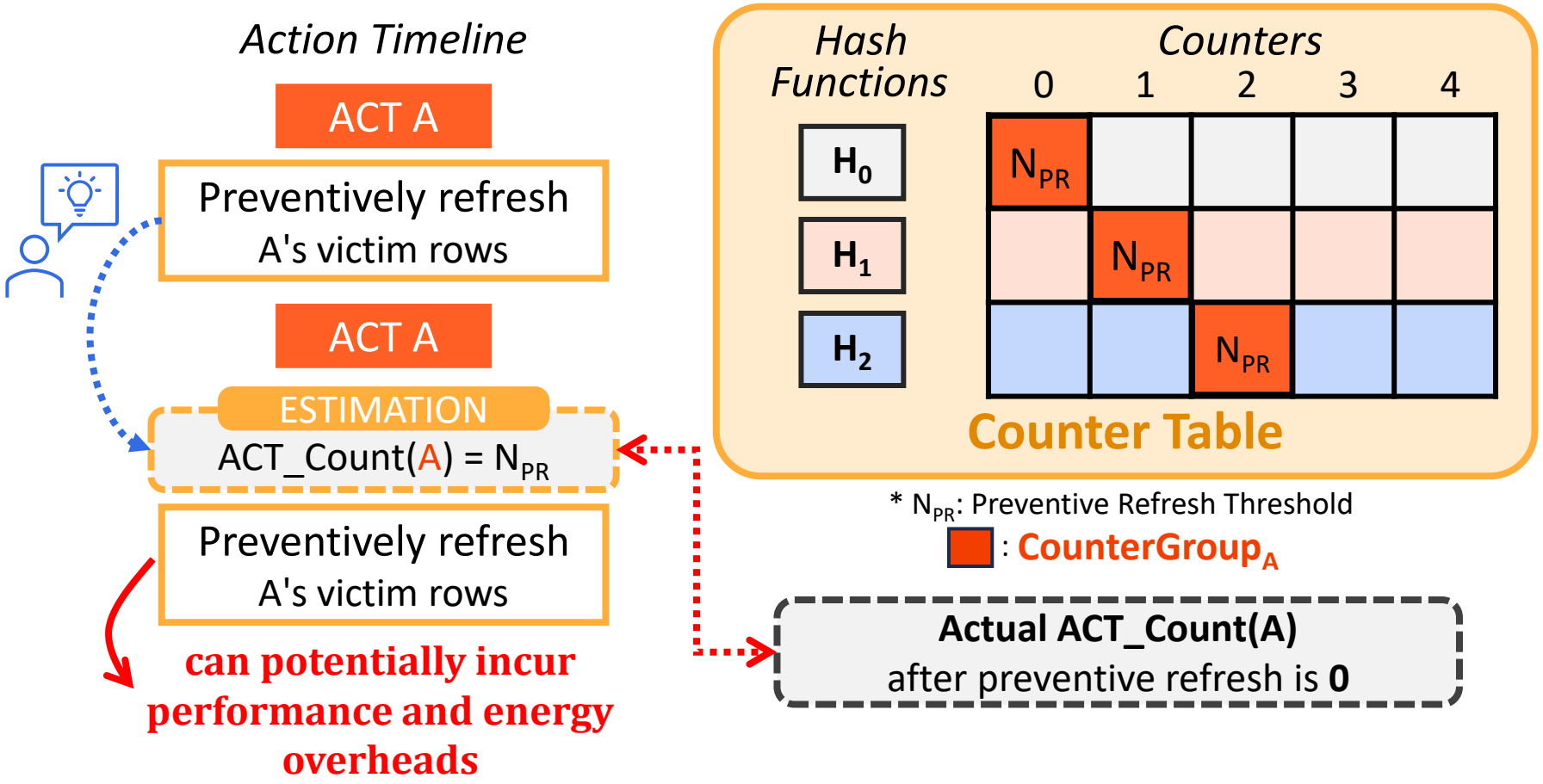
Counter Table (CT): Identifying Aggressor Rows

- CoMeT sets a *preventive refresh threshold (N_{PR})* to timely refresh an aggressor row's victim rows to prevent bitflips



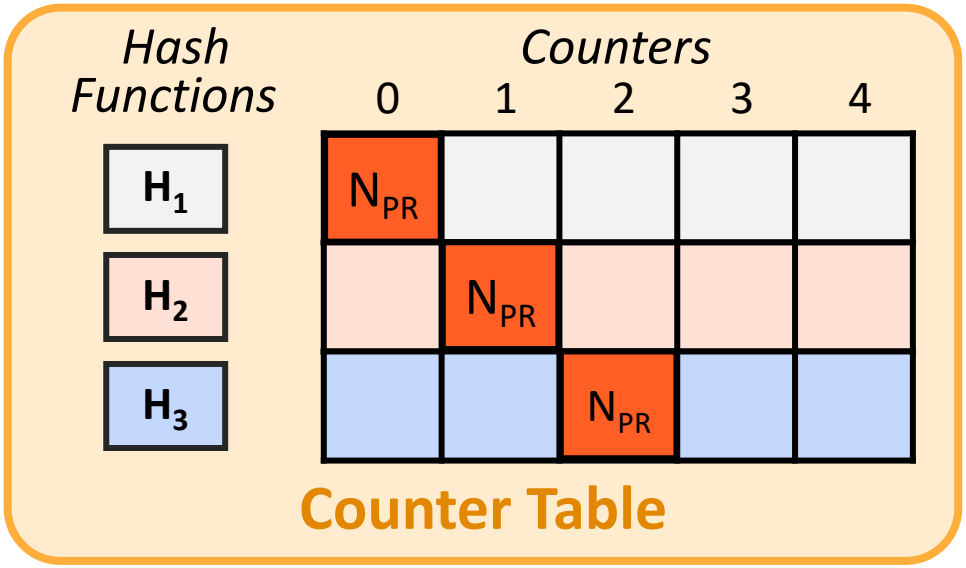
Counter Table (CT): Counter Saturation

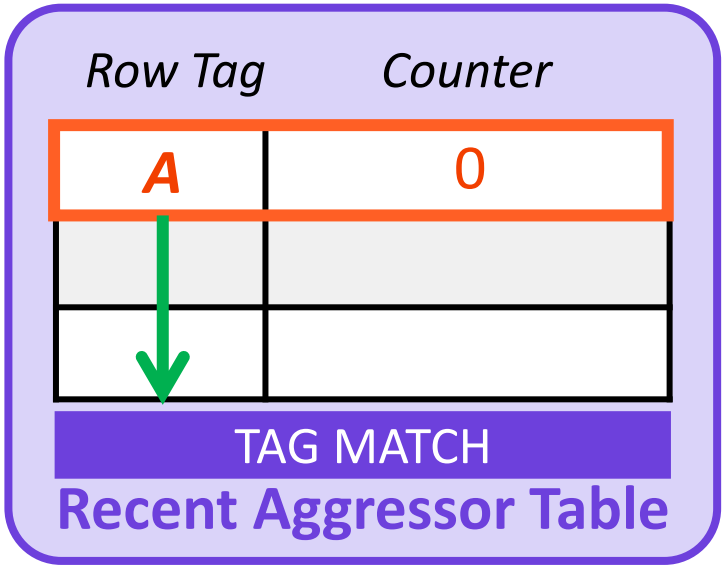
- CoMeT **does not reset** any counter in CT after preventive refresh
  - CT counters saturate at N_{PR}



Recent Aggressor Table

- Allocates per-DRAM-row counters for aggressor rows to accurately estimate their activation counts after preventive refreshes
  - Implemented for *only* a small set of DRAM rows to maintain a low area cost





* N_{PR}: Preventive Refresh Threshold

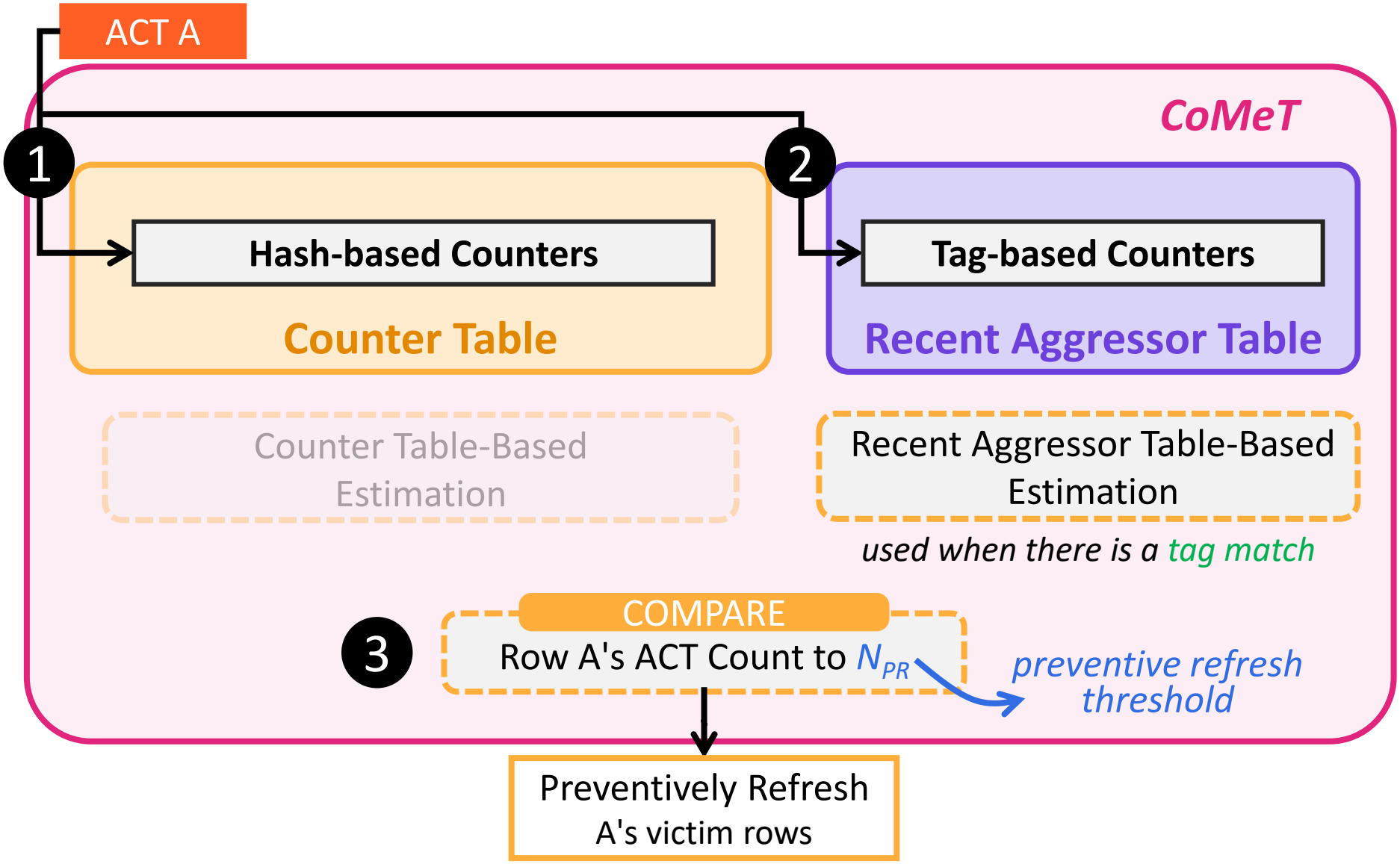
: $CounterGroup_A$

ESTIMATION

ACT_Count(A) = 0

If a DRAM row has a Recent Aggressor Table entry,
CoMeT estimates its activation count 100% accurately

Operation of CoMeT



More Operational Details for CoMeT

- Counter update policy
- Periodic counter reset mechanism
- Recent Aggressor Table eviction policy
- Early preventive refresh at coarse granularity
- Determining the preventive refresh threshold

More Operational Details for CoMeT



CoMeT: Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost

F. Nisa Bostancı İsmail Emir Yüksel Ataberk Olgun Konstantinos Kanellopoulos
Yahya Can Tuğrul A. Giray Yağlıkçı Mohammad Sadrosadati Onur Mutlu

ETH Zürich

DRAM chips are increasingly more vulnerable to read-disturbance phenomena (e.g., RowHammer and RowPress), where repeatedly accessing DRAM rows causes bitflips in nearby rows due to DRAM density scaling. Under low RowHammer thresholds, existing RowHammer mitigations either incur high area overheads or degrade performance significantly.

We propose a new RowHammer mitigation mechanism, CoMeT, that prevents RowHammer bitflips with low area, performance, and energy costs in DRAM-based systems at very

1. Introduction

DRAM chips are susceptible to read-disturbance where repeatedly accessing a DRAM row (i.e., *an aggressor row*) can cause bitflips in physically nearby rows (i.e., *victim rows*) [1–13]. RowHammer is a type of read-disturbance phenomenon that is caused by repeatedly opening and closing (i.e., *hammering*) DRAM rows. Modern DRAM chips become more vulnerable to RowHammer as DRAM technology node size becomes smaller [1, 2, 4, 14–19]: the minimum number of row activations needed to cause a bitflip (i.e., *RowHammer threshold*

https://arxiv.org/abs/2402.18769

https://github.com/CMU-SAFARI/CoMeT

Outline

Background and Problem

Goal and Key Idea

CoMeT: Count-Min-Sketch-based Row Tracking

Evaluation

Conclusion

Evaluation Methodology

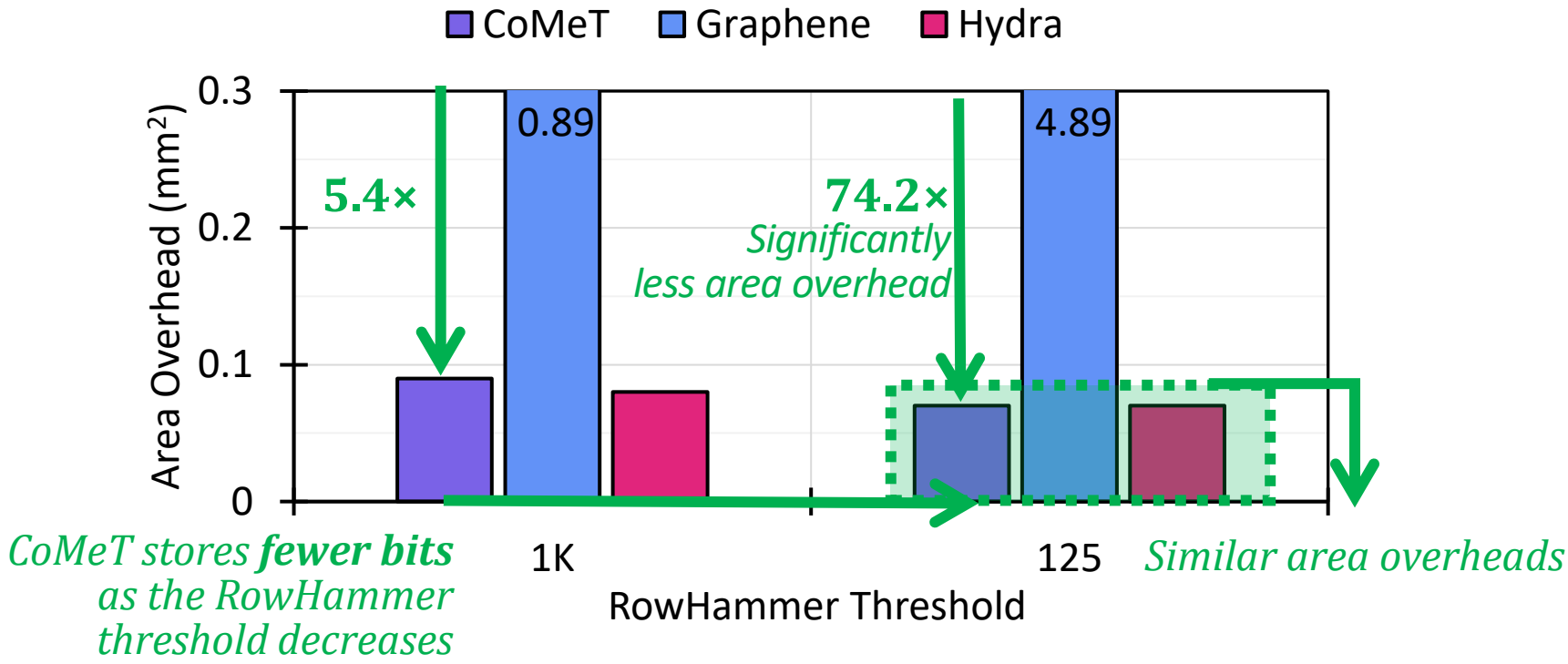
- **Performance and energy consumption evaluation:** cycle-level simulations using Ramulator [Kim+, CAL 2015] and DRAMPower [Chandrasekar+, DATE 2013]
- **System Configuration:**

Processor	1 or 8 cores, 3.6GHz clock frequency, 4-wide issue, 128-entry instruction window
DRAM	DDR4, 1 channel, 2 rank/channel, 4 bank groups, 4 banks/bank group, 128K rows/bank
Memory Ctrl.	64-entry read and write requests queues, Scheduling policy: FR-FCFS with a column cap of 16 Last-Level Cache 8 MiB (single-core), 16 MiB (8-core)
CoMeT	*Counter Table:* 4 hash functions 512 counters per hash *Recent Aggressor Table:* 128 entries

- **Comparison Points:** 4 state-of-the-art RowHammer mitigations
  - Graphene (best performing), Hydra (area-optimized best performing), Low Processor Chip Area Cost: REGA, PARA
- **Workloads:** 61 single-core applications and 56 8-core workload mixes
  - SPEC CPU2006, SPEC CPU2017, TPC, MediaBench, YCSB

Hardware Implementation

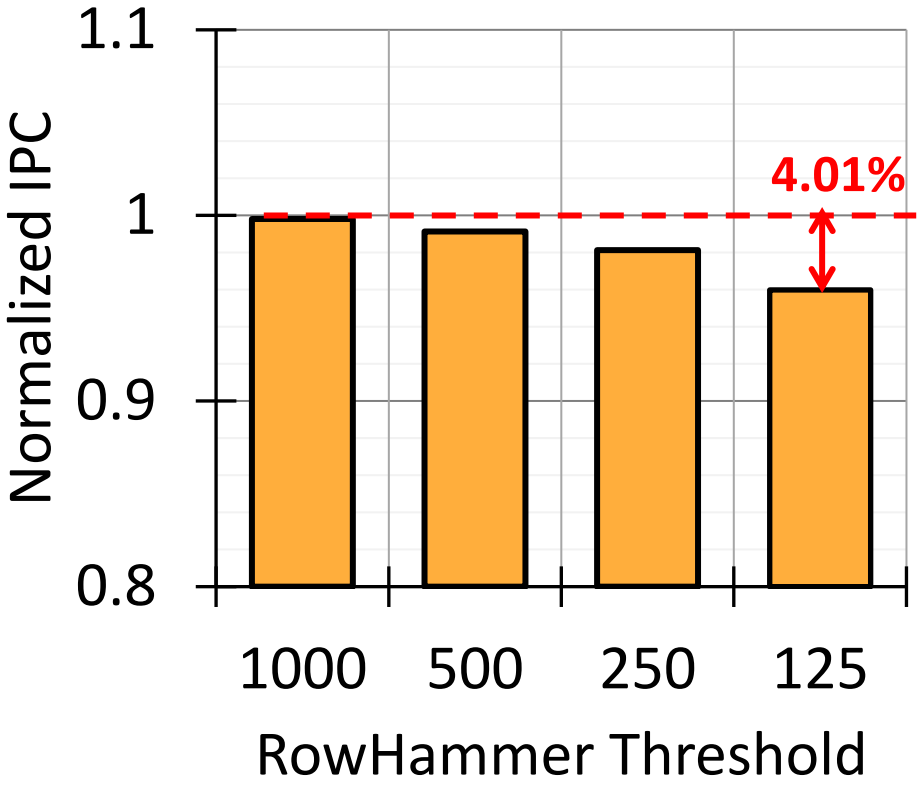
- **Storage and area overhead analysis:** CACTI
- **Dual-rank area overhead comparison:**

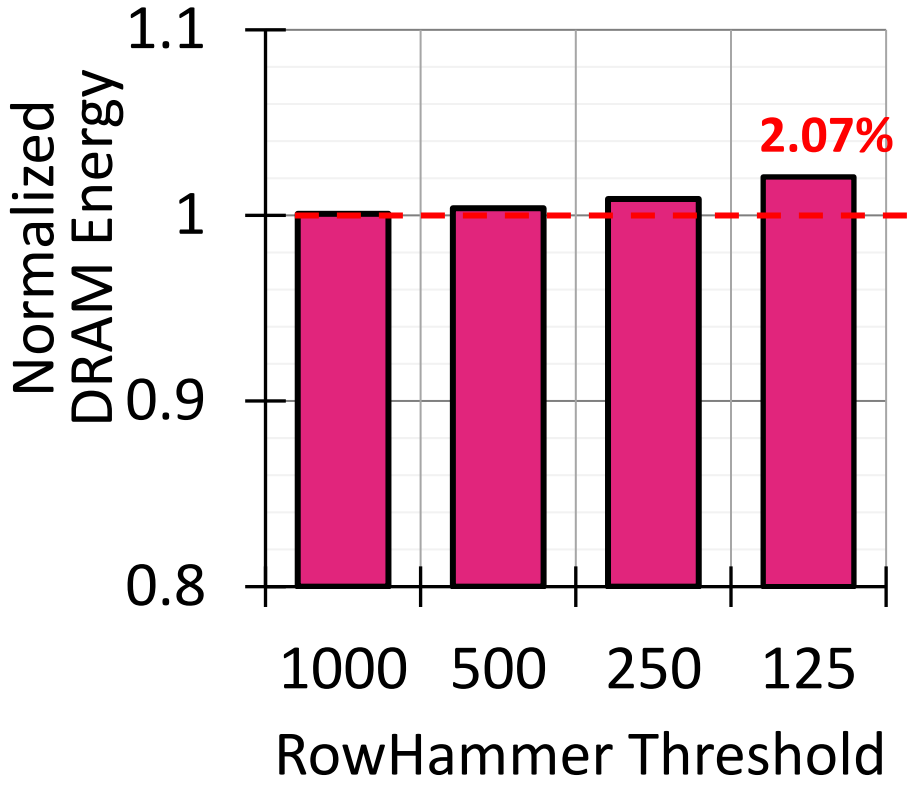


CoMeT incurs a significantly less area overhead than Graphene and a similar area overhead to Hydra

Performance and DRAM Energy

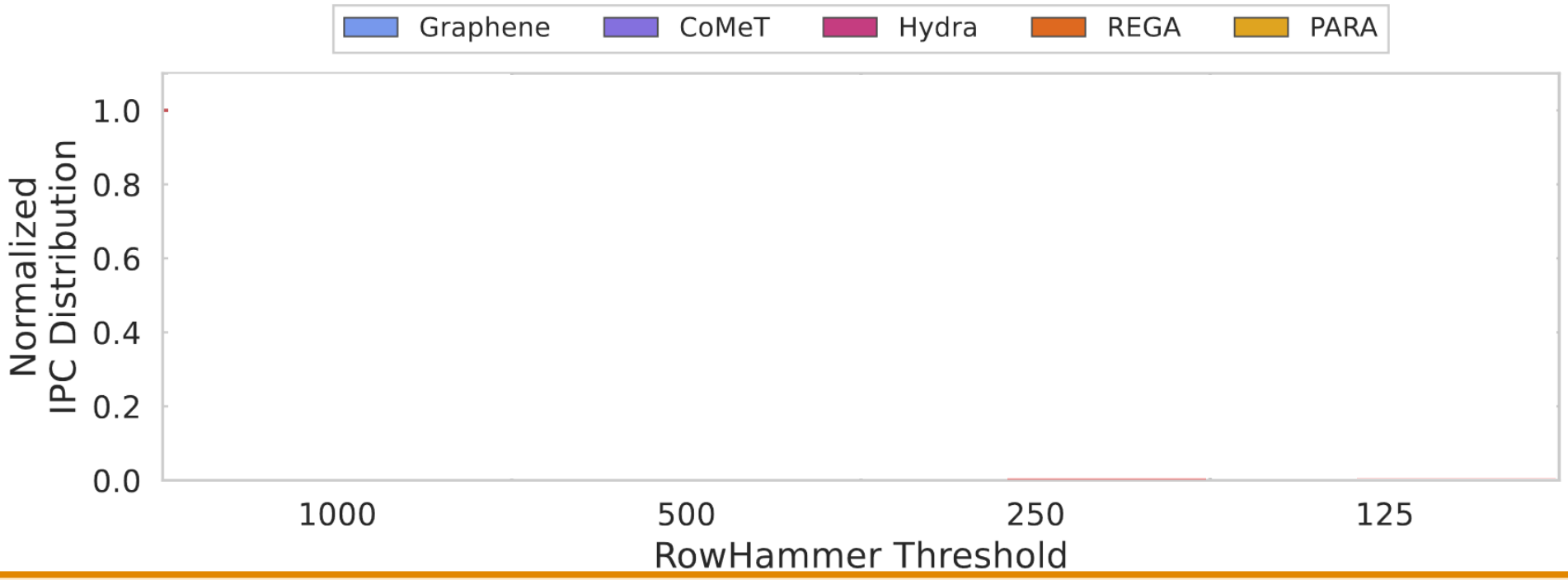
- Average performance and DRAM energy overheads of CoMeT across **single-core applications**





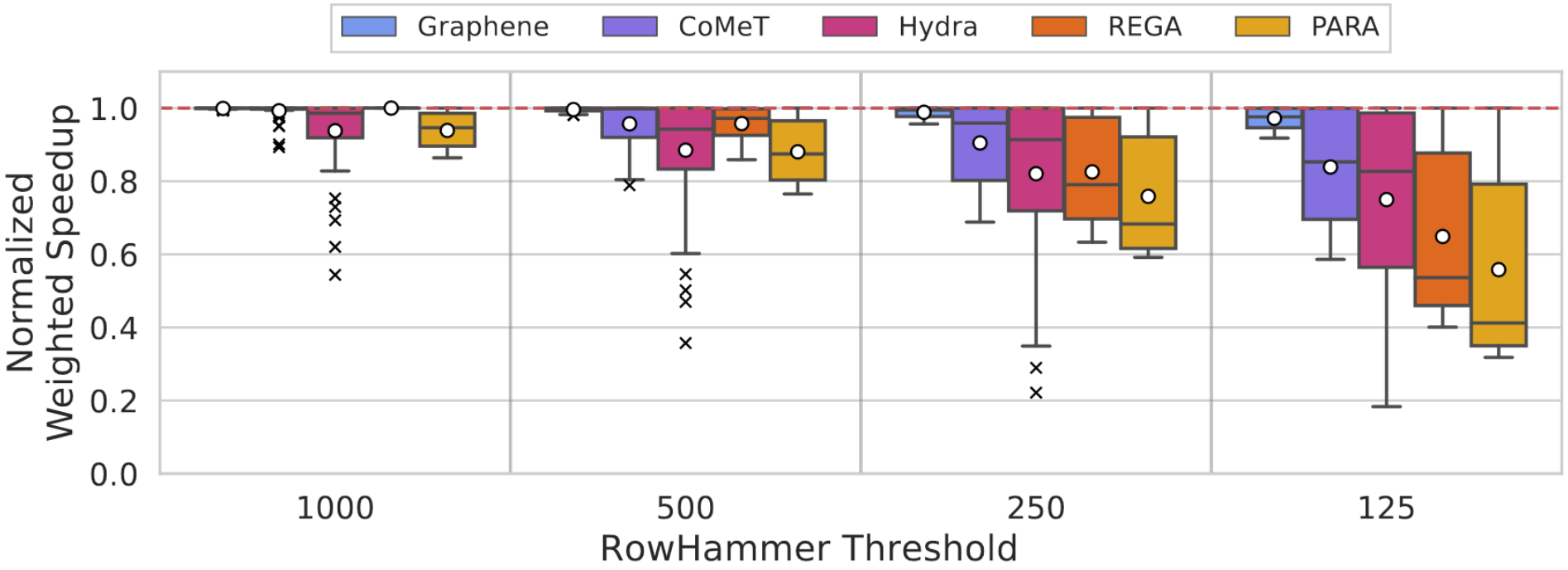
CoMeT prevents bitflips with very small average performance and DRAM energy overheads compared to a baseline system with *no* RowHammer mitigation

Performance Comparison:
Single-Core Applications



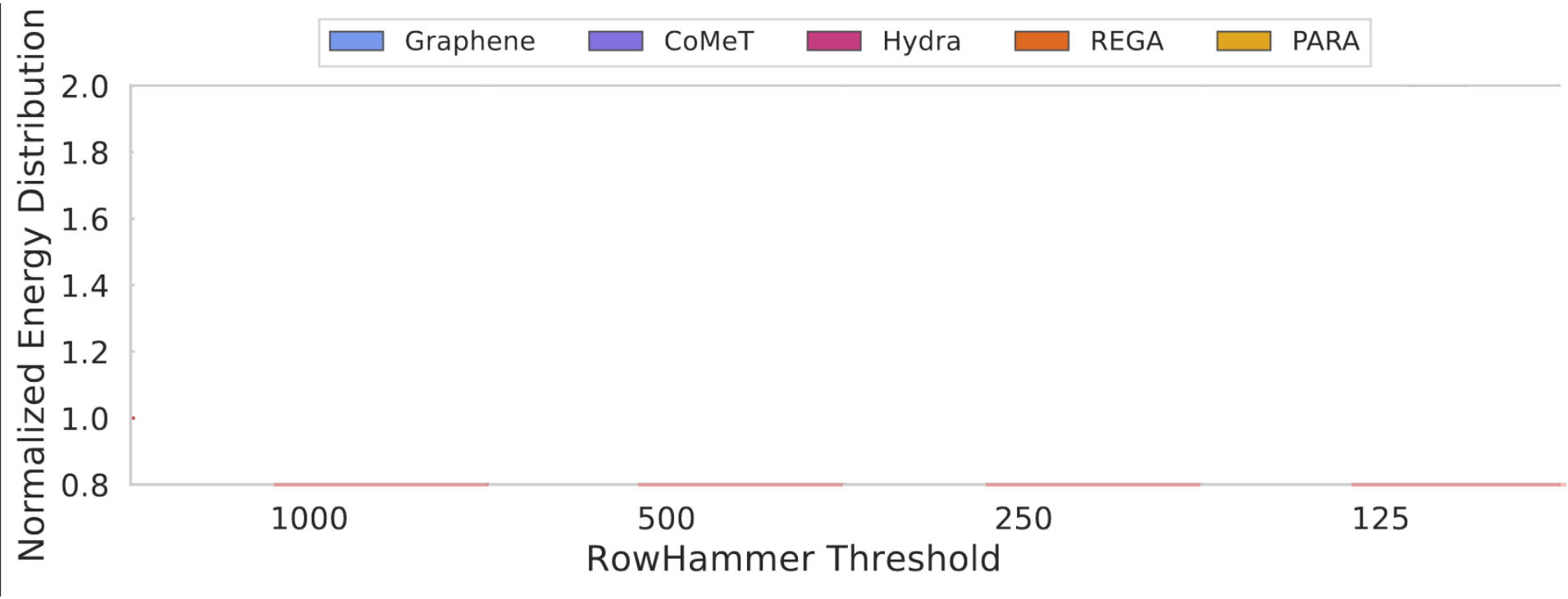
CoMeT incurs a small performance overhead (≤ 1.75%) over Graphene and outperforms Hydra (by up to 39.1%) at all RowHammer thresholds

Performance Comparison:
8-Core Workloads



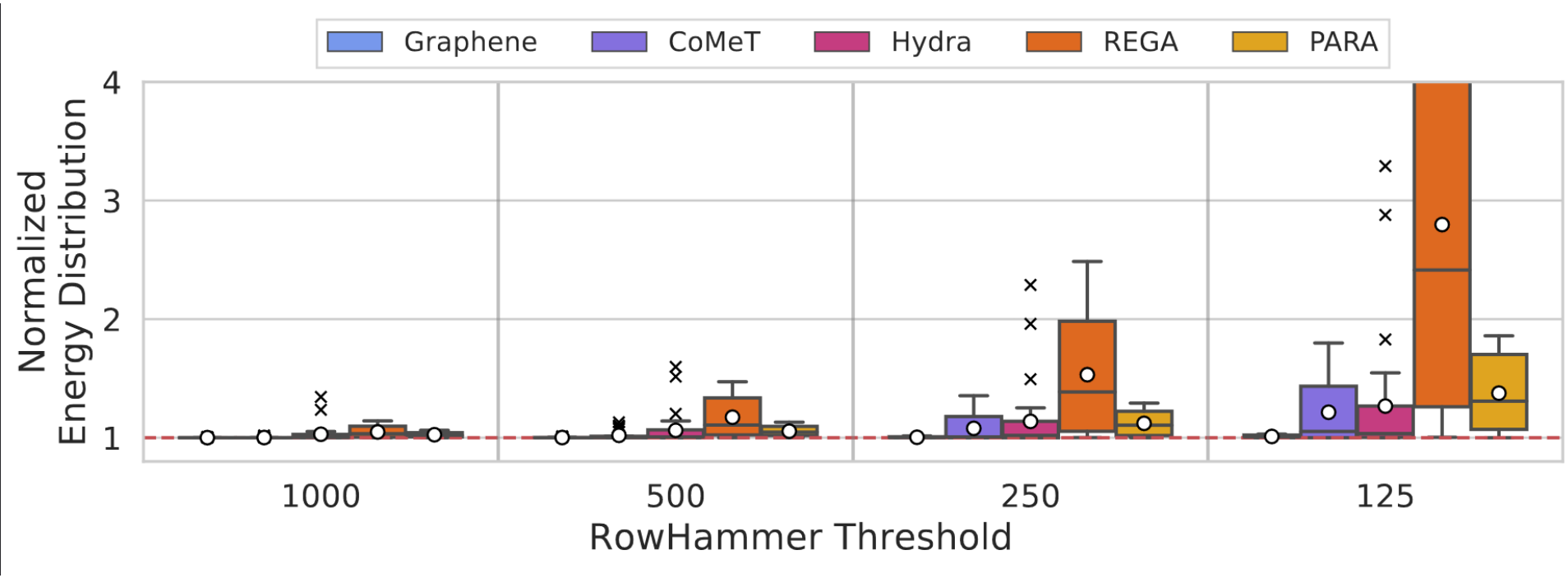
Trends are similar to single-core application evaluation trends

DRAM Energy Comparison:
Single-Core Applications



CoMeT incurs a small DRAM energy overhead (**<1%**) over Graphene and consumes less DRAM energy than Hydra

DRAM Energy Comparison:
8-Core Workloads



Multicore energy trends are similar to single core energy trends

More in the Paper

- Security Analysis of CoMeT
- Sensitivity Analysis
  - Counter Table Configurations
  - Recent Aggressor Table Configurations
  - Counter Reset Period and Preventive Refresh Threshold Values
- CoMeT's Performance under Adversarial Workloads
- Comparison against Throttling-Based Mitigation Techniques
- CoMeT's Performance at High RowHammer Thresholds

More in the Paper



CoMeT: Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost

F. Nisa Bostancı İsmail Emir Yüksel Ataberk Olgun Konstantinos Kanellopoulos
Yahya Can Tuğrul A. Giray Yağlıkçı Mohammad Sadrosadati Onur Mutlu

ETH Zürich

DRAM chips are increasingly more vulnerable to read-disturbance phenomena (e.g., RowHammer and RowPress), where repeatedly accessing DRAM rows causes bitflips in nearby rows due to DRAM density scaling. Under low RowHammer thresholds, existing RowHammer mitigations either incur high area overheads or degrade performance significantly.

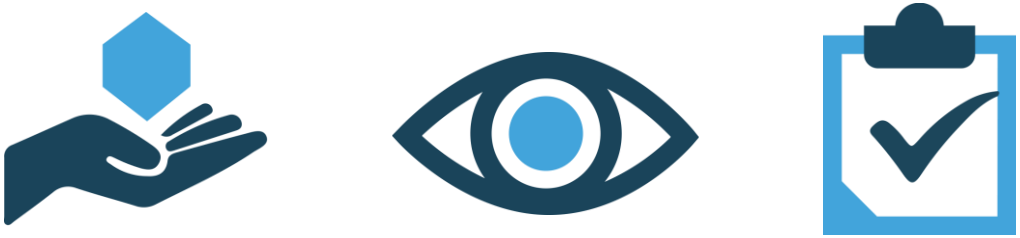
We propose a new RowHammer mitigation mechanism, CoMeT, that prevents RowHammer bitflips with low area, performance, and energy costs in DRAM-based systems at very

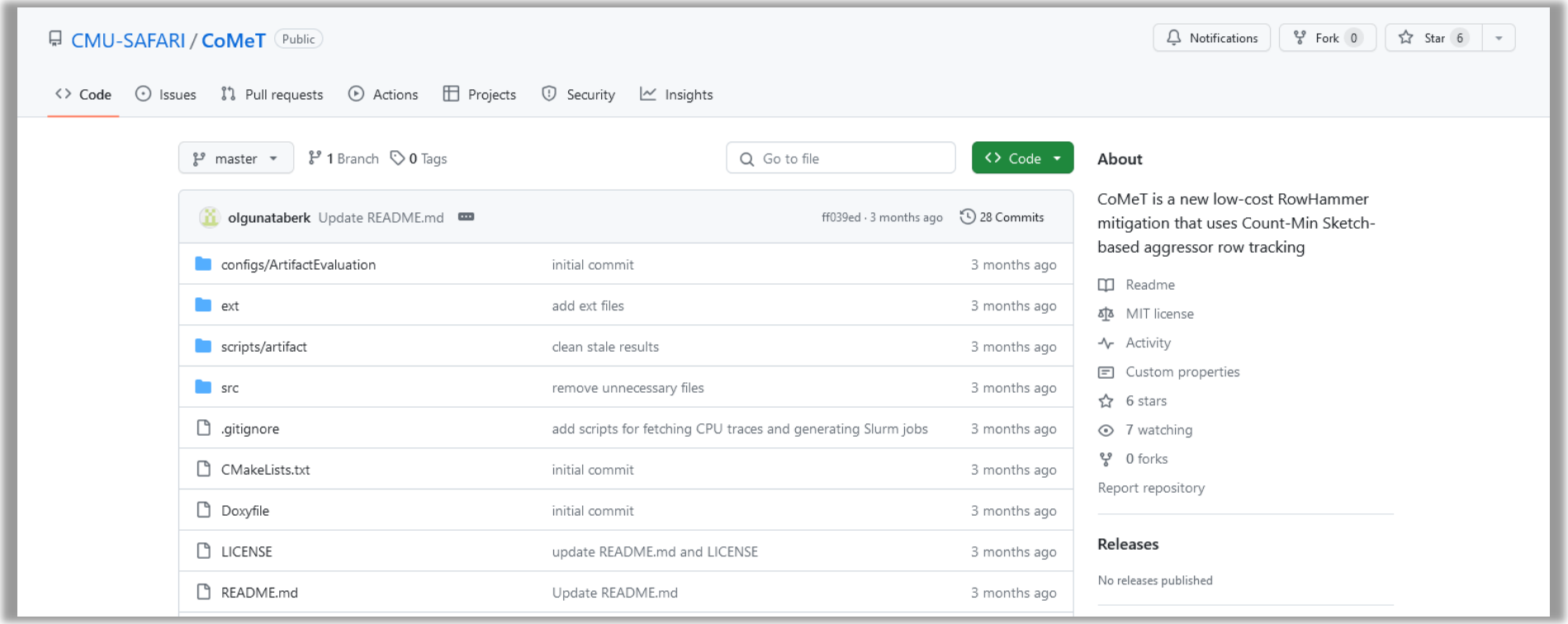
1. Introduction

DRAM chips are susceptible to read-disturbance where repeatedly accessing a DRAM row (i.e., *an aggressor row*) can cause bitflips in physically nearby rows (i.e., *victim rows*) [1–13]. RowHammer is a type of read-disturbance phenomenon that is caused by repeatedly opening and closing (i.e., *hammering*) DRAM rows. Modern DRAM chips become more vulnerable to RowHammer as DRAM technology node size becomes smaller [1, 2, 4, 14–19]: the minimum number of row activations needed to cause a bitflip (i.e., *RowHammer threshold*

https://arxiv.org/abs/2402.18769

CoMeT is Open Source and Artifact Evaluated





https://github.com/CMU-SAFARI/CoMeT

Outline

Background and Problem

Goal and Key Idea

CoMeT: Count-Min-Sketch-based Row Tracking

Evaluation

Conclusion

Conclusion

Goal: Prevent RowHammer bitflips with low area, performance, and energy overheads in highly RowHammer-vulnerable DRAM-based systems

Key Idea: Use low-cost and scalable hash-based counters to accurately track DRAM rows

CoMeT:
- tracks most DRAM rows with scalable hash-based counters by employing the Count-Min-Sketch technique to achieve a low area cost
- tracks only a small set of DRAM rows that are activated many times with highly accurate per-DRAM-row activation counters to reduce performance penalties

Evaluation: CoMeT achieves a good trade-off between area, performance and energy costs
- incurs significantly less area overhead (74.2×) compared to the state-of-the-art technique
- outperforms the state-of-the-art technique (by up to 39.1%)

https://github.com/CMU-SAFARI/CoMeT



CoMeT

Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost

F. Nisa Bostancı

I. E. Yüksel A. Olgun K. Kanellopoulos Y. C. Tuğrul

A. G. Yağlıkçı M. Sadrosadati O. Mutlu

Paper Link

GitHub Repo

SAFARI

ETH zürich



CoMeT

Count-Min-Sketch-based Row Tracking to Mitigate RowHammer at Low Cost

BACKUP SLIDES

F. Nisa Bostancı

I. E. Yüksel A. Olgun K. Kanellopoulos Y. C. Tuğrul

A. G. Yağlıkçı M. Sadrosadati O. Mutlu

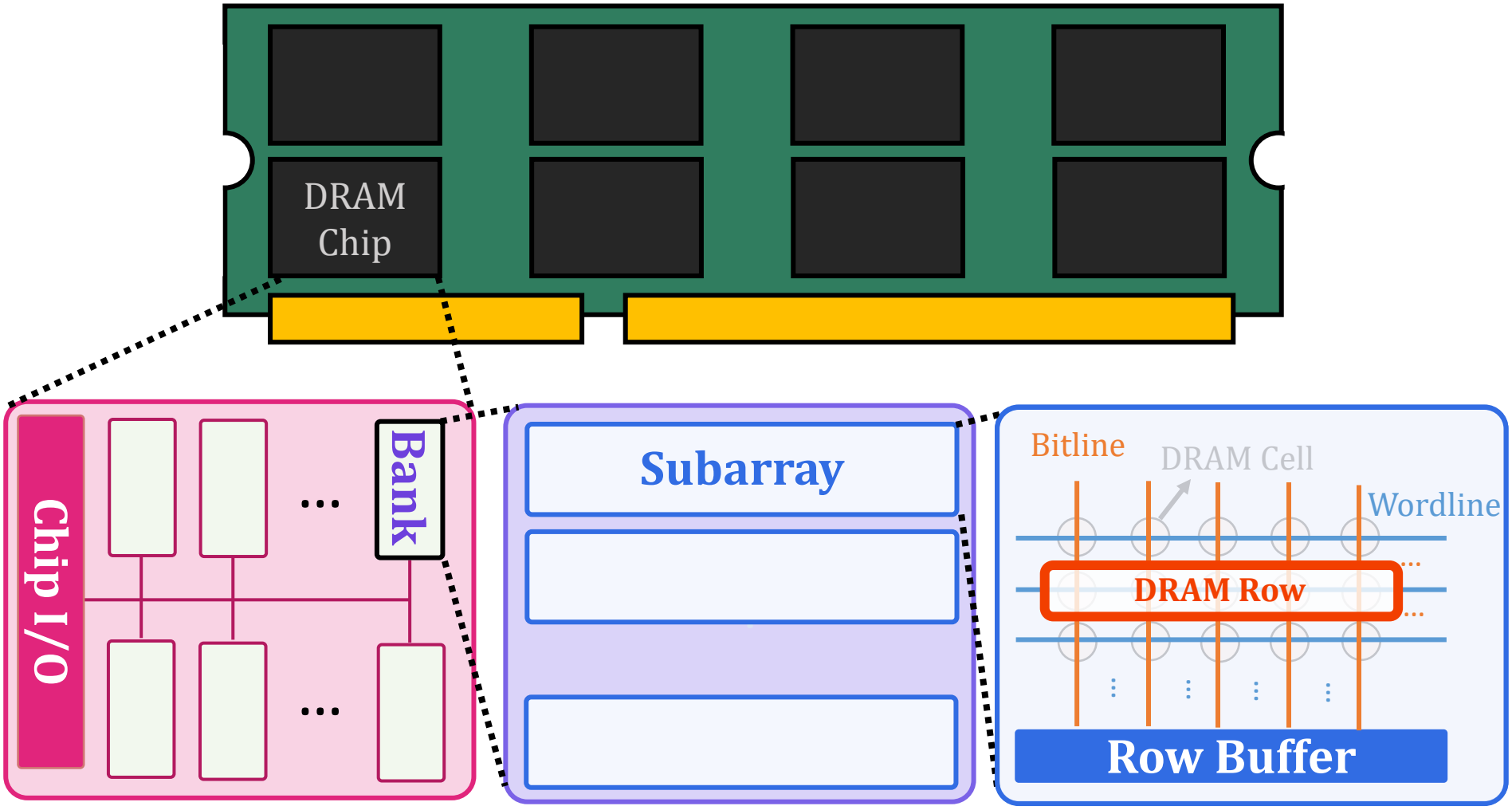
Paper Link

GitHub Repo

SAFARI

ETH zürich

DRAM Organization



Read Disturbance Vulnerabilities

DRAM Subarray

[Kim+ ISCA'20]

The minimum number of row activations needed to cause a bitflip (i.e., *RowHammer threshold* (N_{RH})) has **reduced by more than an order of magnitude** in less than a decade

closed

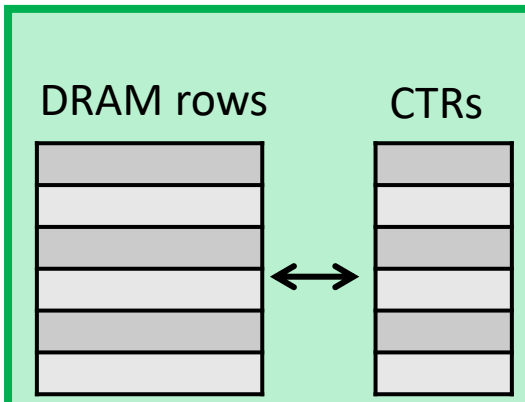
Row 2

[Luo+ ISCA'23]

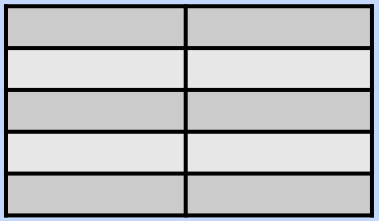
RowPress is shown to lead to bitflips with **one to two orders of magnitude fewer activations** (than RowHammer) under realistic conditions

Repeatedly **opening** (activating) and **closing** (precharging) a DRAM row causes **RowHammer bitflips** in nearby cells

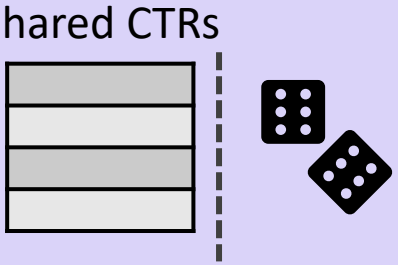
Existing RowHammer Mitigations (II): Overview of Mitigation Techniques



One ACT counter per **DRAM row**



One ACT Counter per **potential aggressor row**



<1 counter per **DRAM row**

Performance Overhead	VERY LOW	LOW	HIGH
Area/Storage Overhead	VERY HIGH	HIGH	LOW

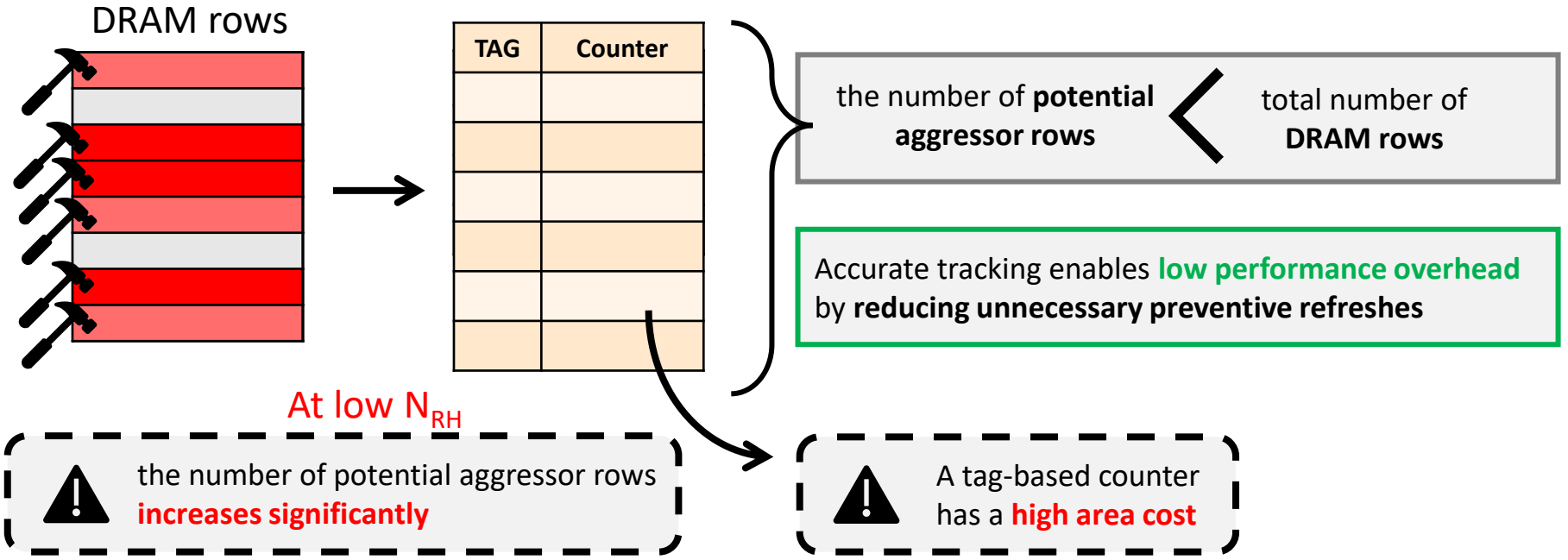
~20 MiB for 2^{23} rows with 10-bit counters

Increasing number of aggressor rows + expensive structures

Low accuracy + Increasing usage of DRAM bandwidth

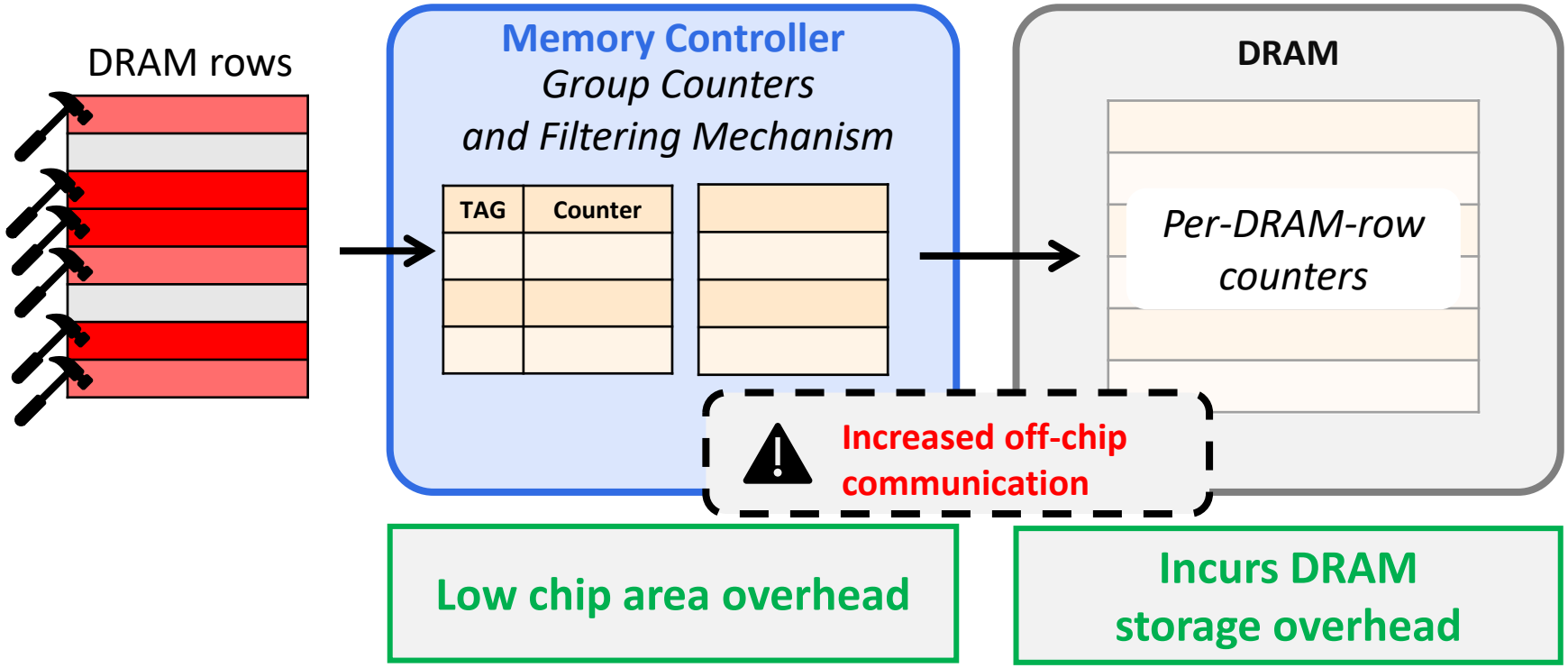
Existing RowHammer Mitigations (II): Performance Optimized Mitigations

- Accurately tracking DRAM row activations can be done by allocating per-row counters to potential aggressor rows



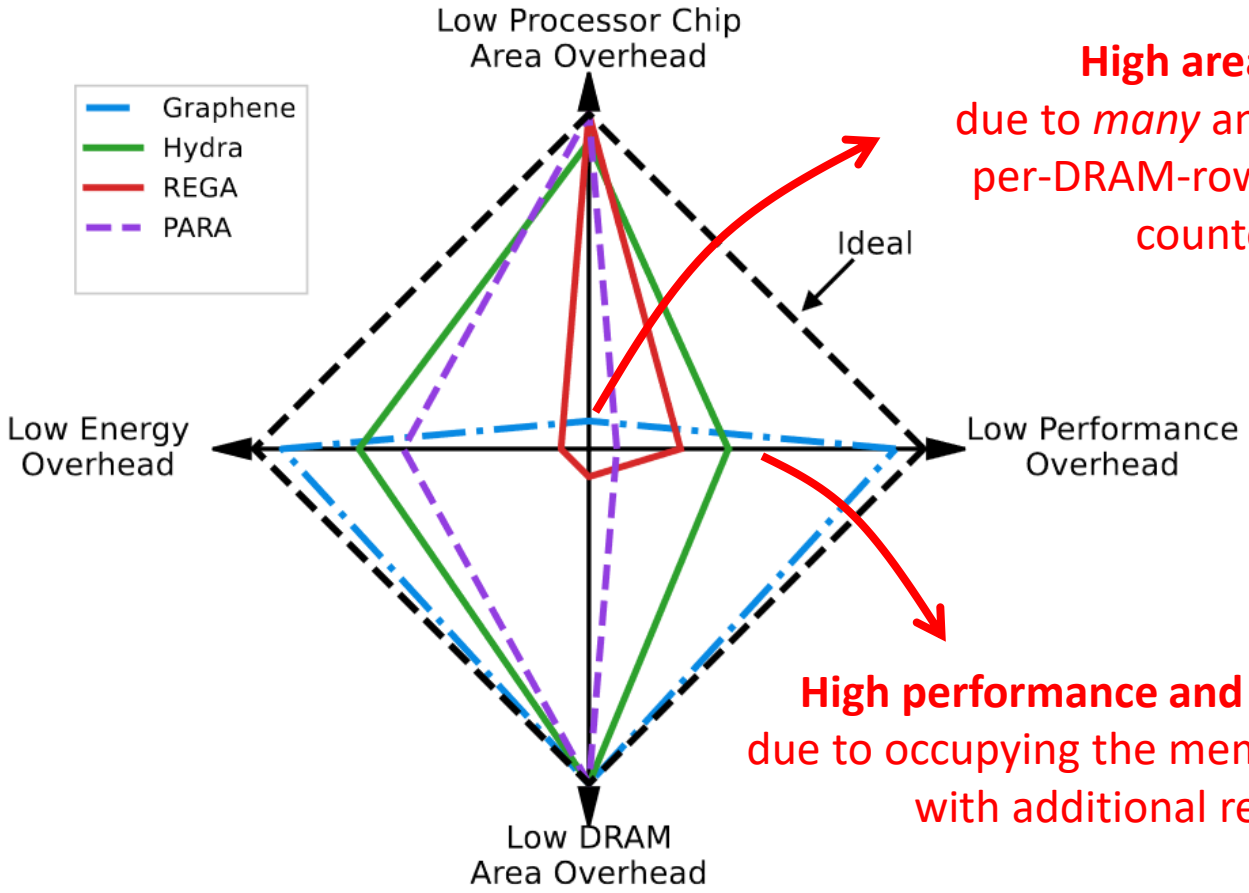
As DRAM becomes more vulnerable to read disturbance, tracking all potential aggressor rows with tag-based counters results in **a high area overhead**

Existing RowHammer Mitigations (III): Area Optimized Mitigations: Hydra



As DRAM becomes more vulnerable to read disturbance, increased off-chip communication results in **high performance and energy overheads**

Limitations of Existing Mitigations

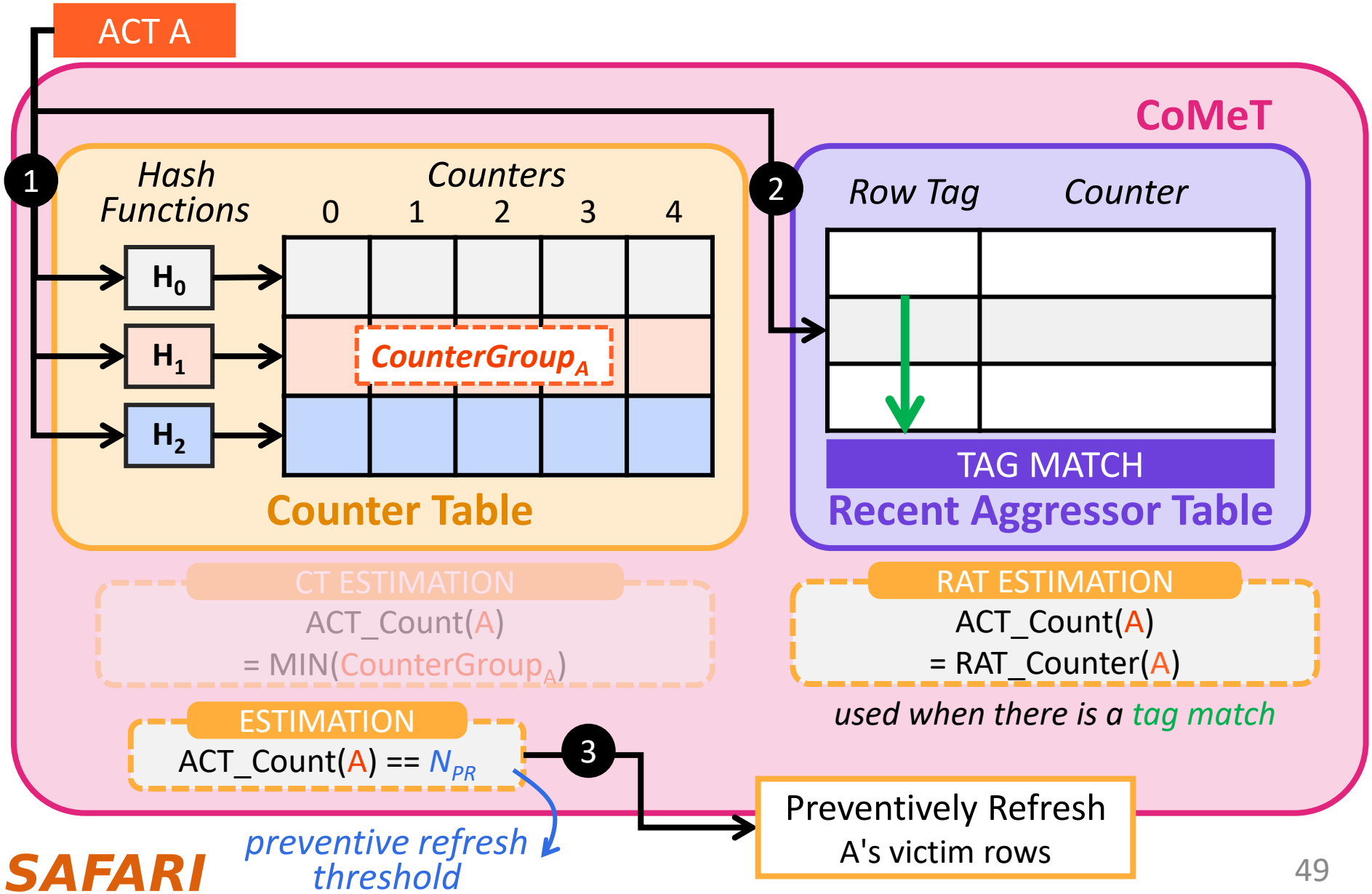


High area cost
due to *many* and *expensive* per-DRAM-row activation counters

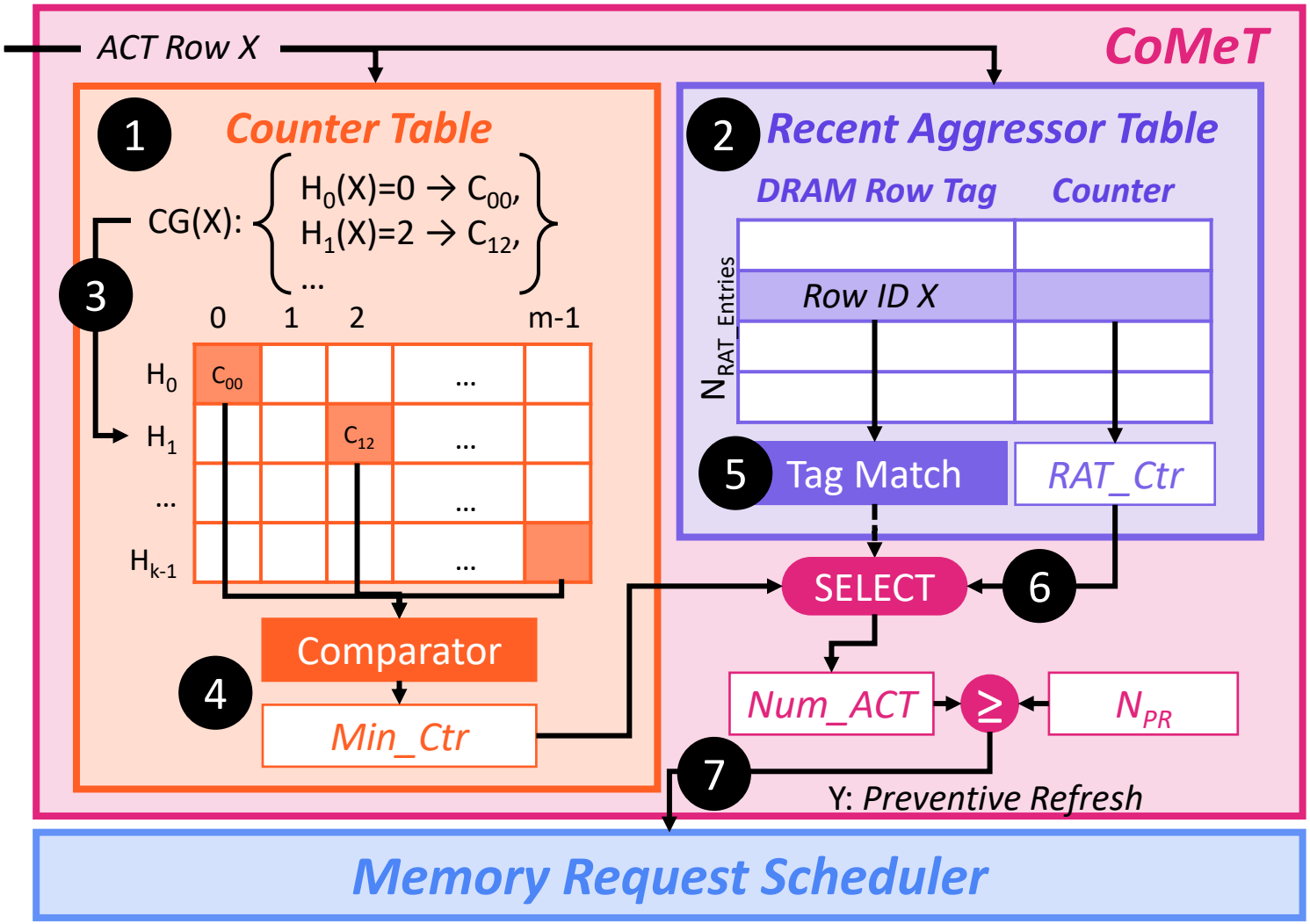
High performance and energy costs
due to occupying the memory bandwidth with additional requests

No existing mitigation technique prevents RowHammer bitflips
at low area, performance and energy costs

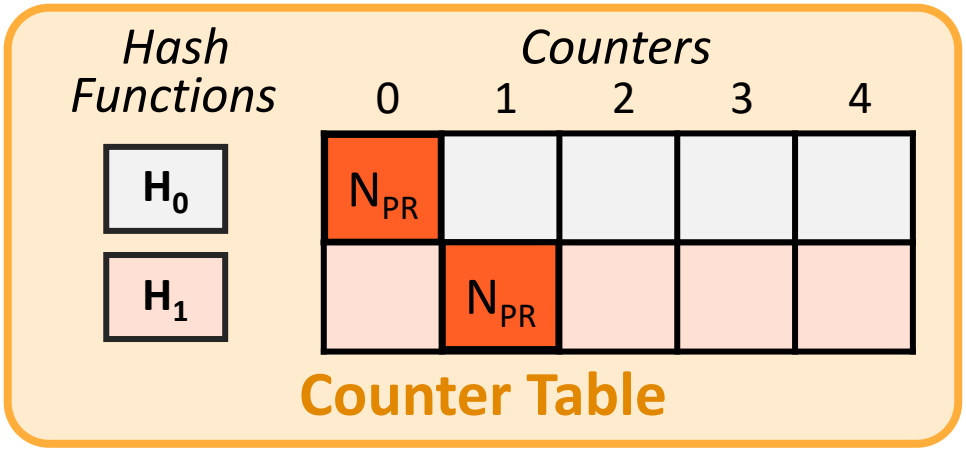
Operation of CoMeT

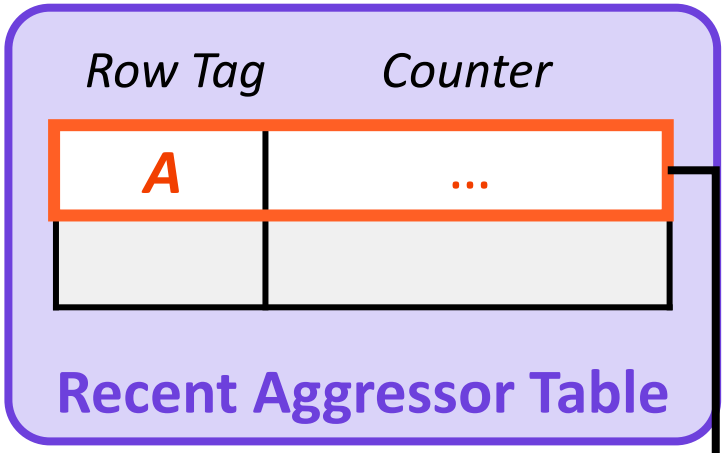


CoMeT Overview



Early Preventive Refresh at Coarse Granularity





* N_{PR}: Preventive Refresh Threshold

: CounterGroup_A

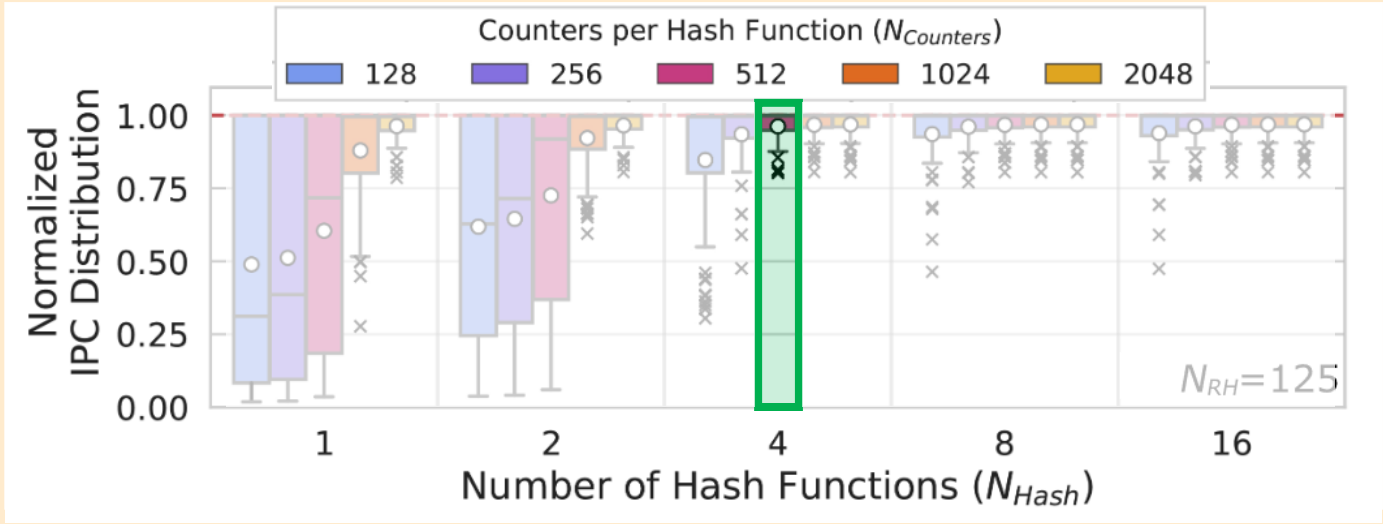
1 Entries can be evicted if there are many aggressor rows

2 CoMeT falls back to Counter Table estimation (N_{PR})

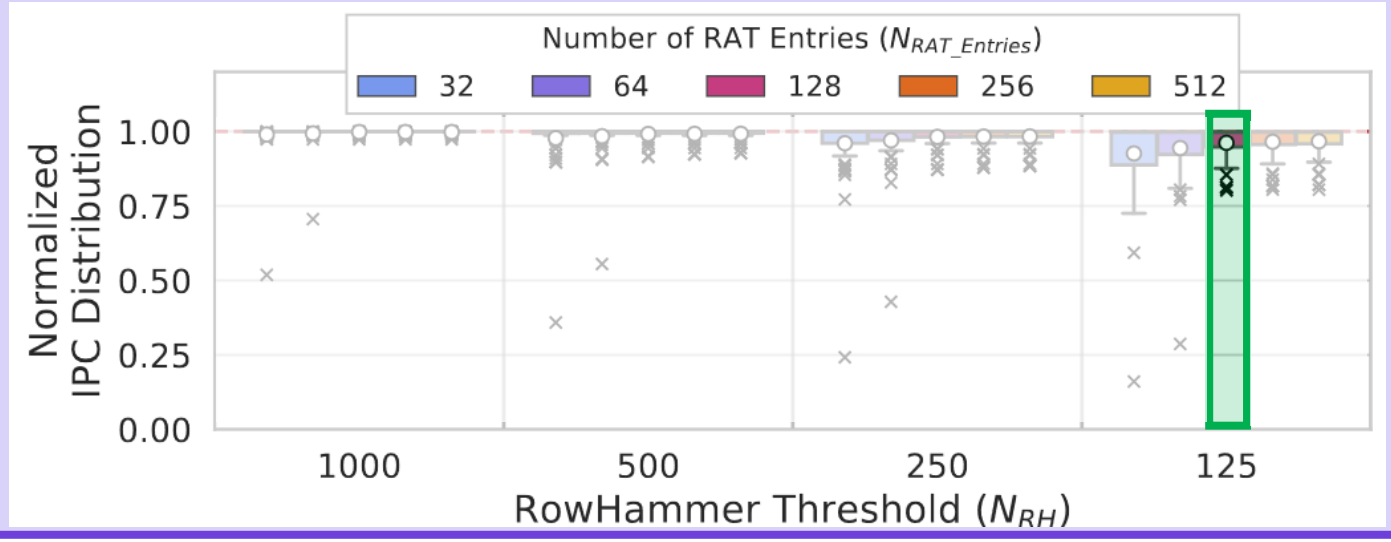
3 Preventively refreshes victim rows

Configuring CoMeT – Sensitivity Analysis:
Counter Table and Recent Aggressor Table

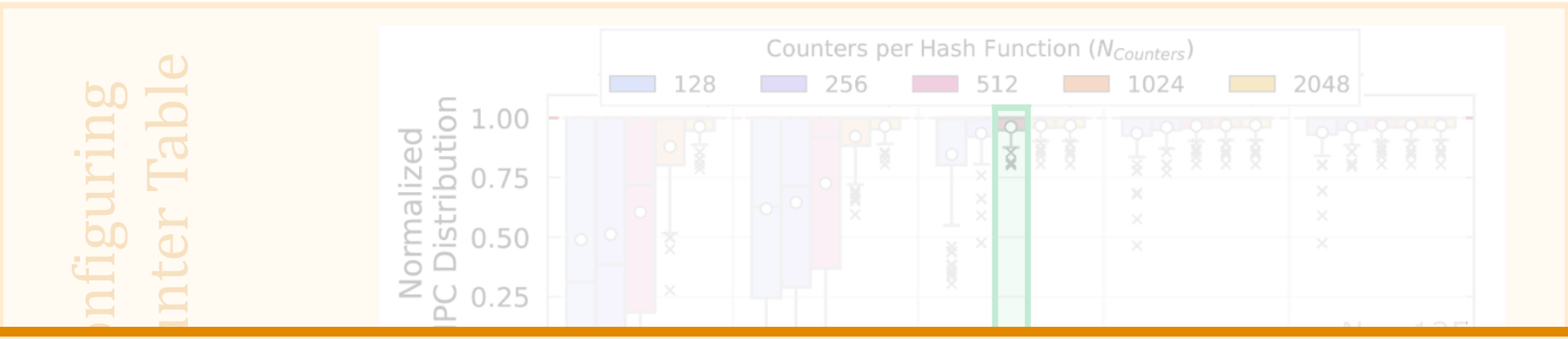
Configuring Counter Table



Configuring RAT

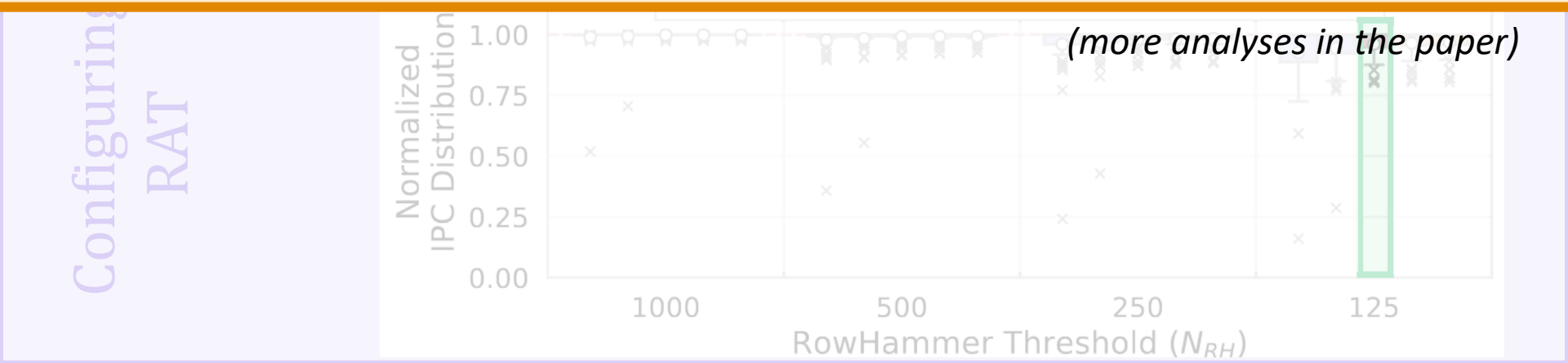


Configuring CoMeT – Sensitivity Analysis:
Counter Table and Recent Aggressor Table



CoMeT configuration that achieves both performance and area efficiency:
Counter Table with **4 hash functions** and **512 counters per hash function**
Recent Aggressor Table with **128 entry**

(more analyses in the paper)



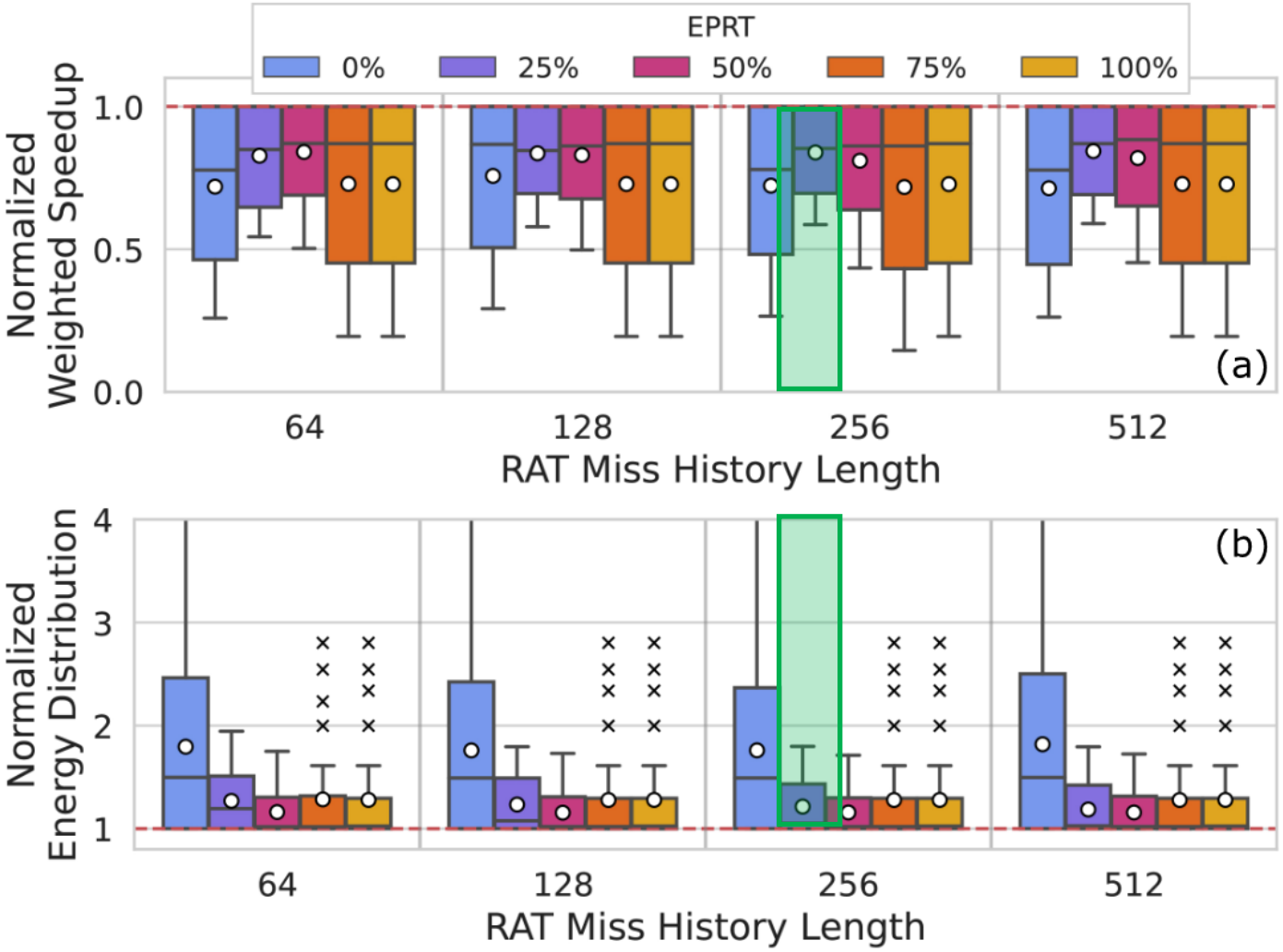
Configuring CoMeT – Sensitivity Analysis: Counter Reset Period and N_{PR}

$$Counter\ Refresh\ Period = \frac{tREFW}{k} \quad (1)$$

$$N_{PR} = \frac{N_{RH}}{k+1} \quad (2)$$



The effect of EPRT and RAT Miss History Length on Performance and DRAM energy consumption



Hardware Implementation

- **Storage and area overhead analysis:** CACTI
  - Logic circuitry overhead: Verilog HDL implementation and Synopsys DC
- **Dual-rank area overhead comparison:**

	N_{RH}=1K		N_{RH}=500		N_{RH}=250		N_{RH}=125	
	KB	mm^2	KB	mm^2	KB	mm^2	KB	mm^2
CoMeT	76.5	0.09	68.0	0.08	59.5	0.07	51.0	0.07
CT (SRAM)	64.0	0.05	56.0	0.05	48.0	0.04	40.0	0.04
RAT (CAM)	12.5	0.03	12.0	0.03	11.5	0.03	11.0	0.02
Logic Circuitry	-	0.005	-	0.005	-	0.005	-	0.005
Graphene [86]	207.2	0.49	398.4	1.13	765.0	3.01	1466.2	4.89
Hydra [90][8]	61.6	0.08	56.5	0.08	51.4	0.07	46.8	0.07

As N_{RH} decreases, CoMeT's area and storage overheads decrease due to storing fewer bits for its counters

Hardware Implementation

- **Storage and area overhead analysis:** CACTI
  - Logic circuitry overhead: Verilog HDL implementation and Synopsys DC
- **Dual-rank area overhead comparison:**

	N_{RH}=1K		N_{RH}=500		N_{RH}=250		N_{RH}=125	
	KB	mm^2	KB	mm^2	KB	mm^2	KB	mm^2
CoMeT	76.5	0.09	68.0	0.08	59.5	0.07	51.0	0.07
CT (SRAM)	64.0	0.05	56.0	0.05	48.0	0.04	40.0	0.04
RAT (CAM)	12.5	0.03	12.0	0.03	11.5	0.03	11.0	0.02
Logic Circuitry	-	0.005	-	0.005	-	0.005	-	0.005
Graphene [86]	207.2	0.49	398.4	1.13	765.0	3.01	1466.2	4.89
Hydra [90][8]	61.6	0.08	56.5	0.08	51.4	0.07	46.8	0.07

Compared to the best performing state-of-the-art mitigation, CoMeT induces significantly less area overhead ()

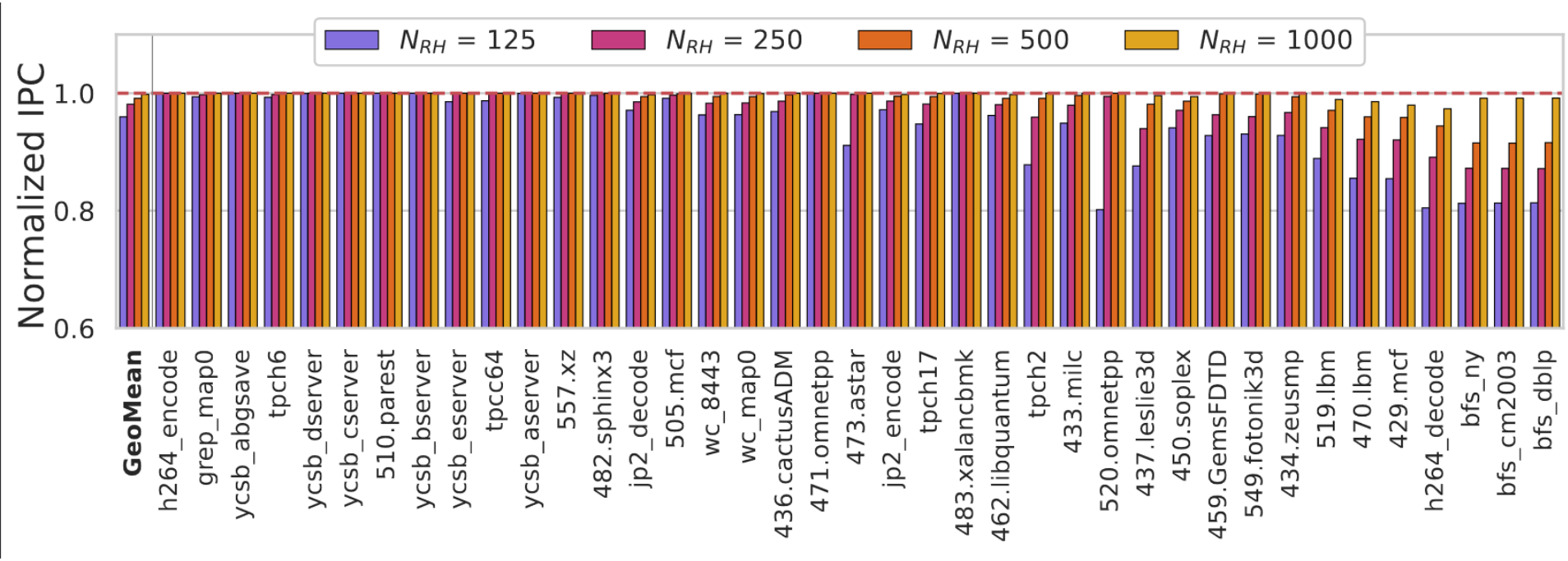
Hardware Implementation

- **Storage and area overhead analysis:** CACTI
  - Logic circuitry overhead: Verilog HDL implementation and Synopsys DC
- **Dual-rank area overhead comparison:**

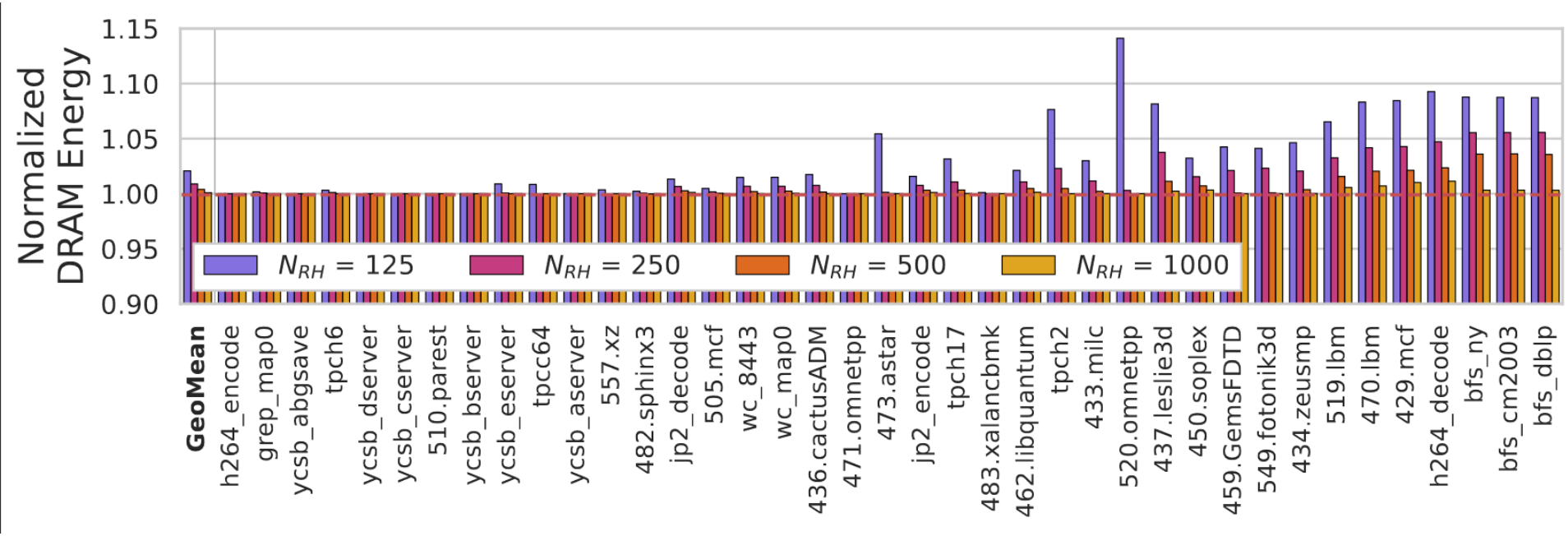
	N_{RH}=1K		N_{RH}=500		N_{RH}=250		N_{RH}=125	
	KB	mm^2	KB	mm^2	KB	mm^2	KB	mm^2
CoMeT	76.5	0.09	68.0	0.08	59.5	0.07	51.0	0.07
CT (SRAM)	64.0	0.05	56.0	0.05	48.0	0.04	40.0	0.04
RAT (CAM)	12.5	0.03	12.0	0.03	11.5	0.03	11.0	0.02
Logic Circuitry	-	0.005	-	0.005	-	0.005	-	0.005
Graphene [86]	207.2	0.49	398.4	1.13	765.0	3.01	1466.2	4.89
Hydra [90][8]	61.6	0.08	56.5	0.08	51.4	0.07	46.8	0.07

Compared to the best performing low-area-cost mitigation, CoMeT induces similar area overhead

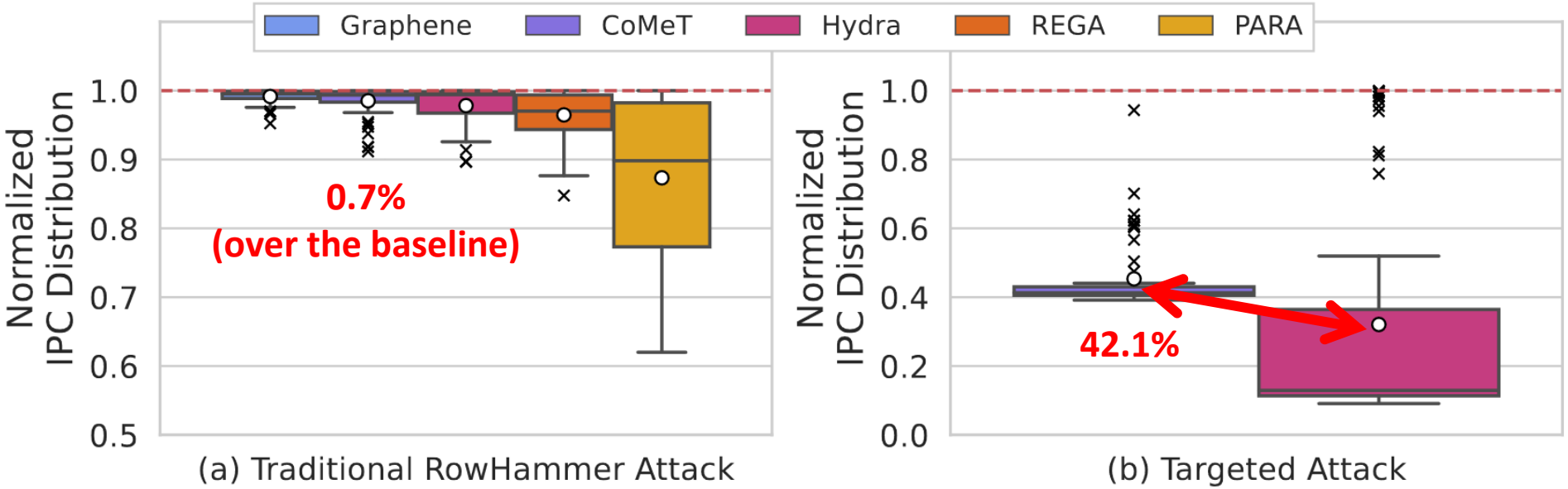
Single-Core Performance
Workload Breakdown (Medium and High Intensity)



Single-Core DRAM Energy
Workload Breakdown (Medium and High Intensity)



Adversarial Access Patterns



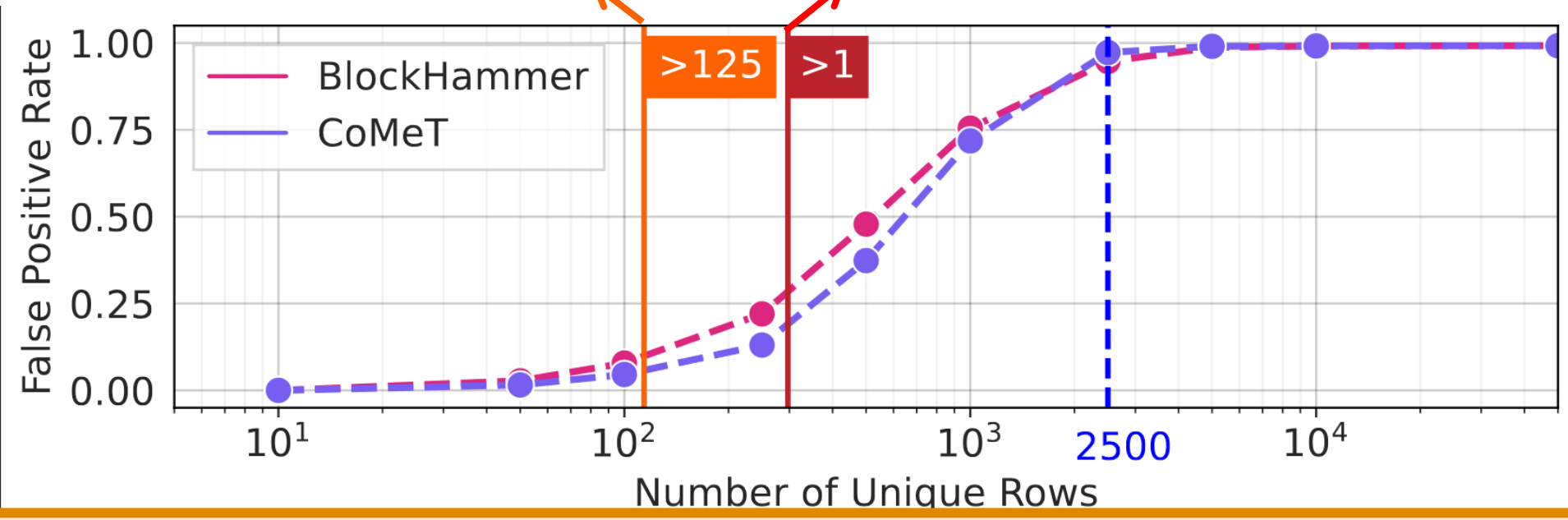
CoMeT incurs negligible additional performance overhead on benign workloads when a traditional RowHammer attack is running at the same time

Comparison Against BlockHammer (I)
Tracker Comparison

- CoMeT and BlockHammer employs different algorithms and this results in different DRAM-row-to-counter mappings

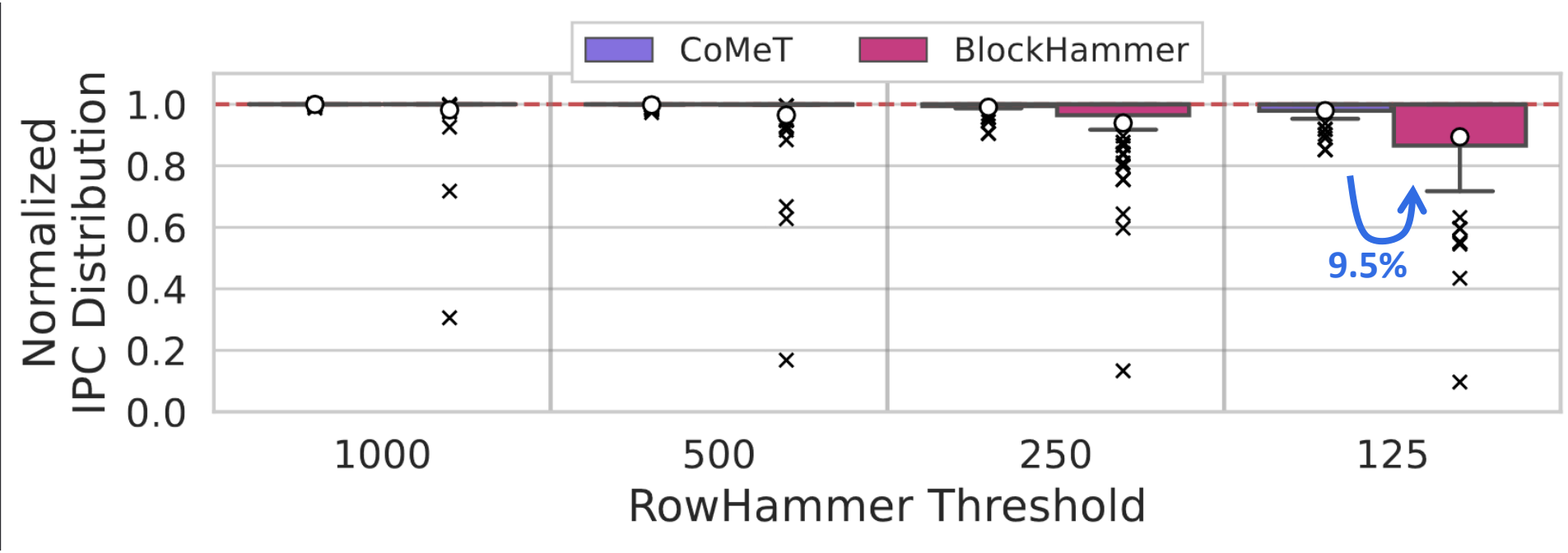
Average # of unique rows that are activated **at least 125 times** by **benign workloads**

the average number of unique rows touched **at least once** by **benign workloads**



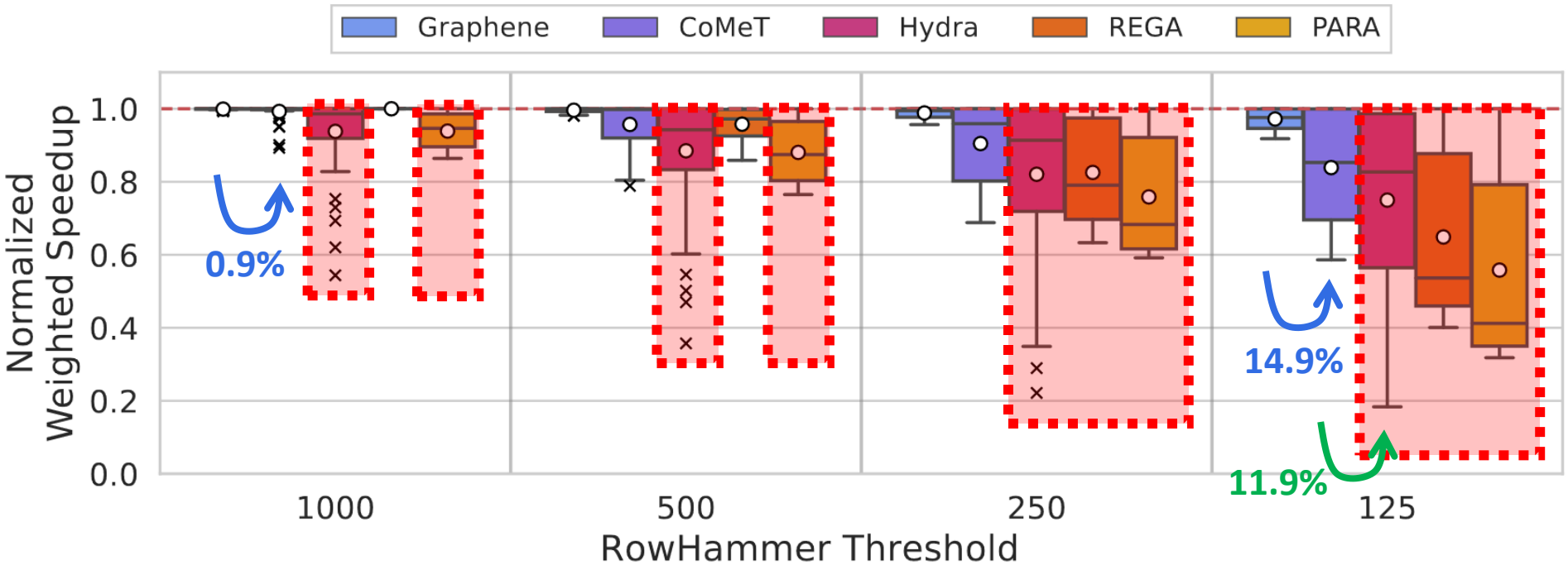
When tracking at most 2,500 unique rows, CoMeT's tracker outperforms BlockHammer

Comparison Against BlockHammer (II)
Single-Core Performance Comparison



CoMeT outperforms BlockHammer due to BlockHammer's (i) high false positive rate and (ii) increased memory request latencies due to throttling

Performance Comparison:
8-Core Workloads

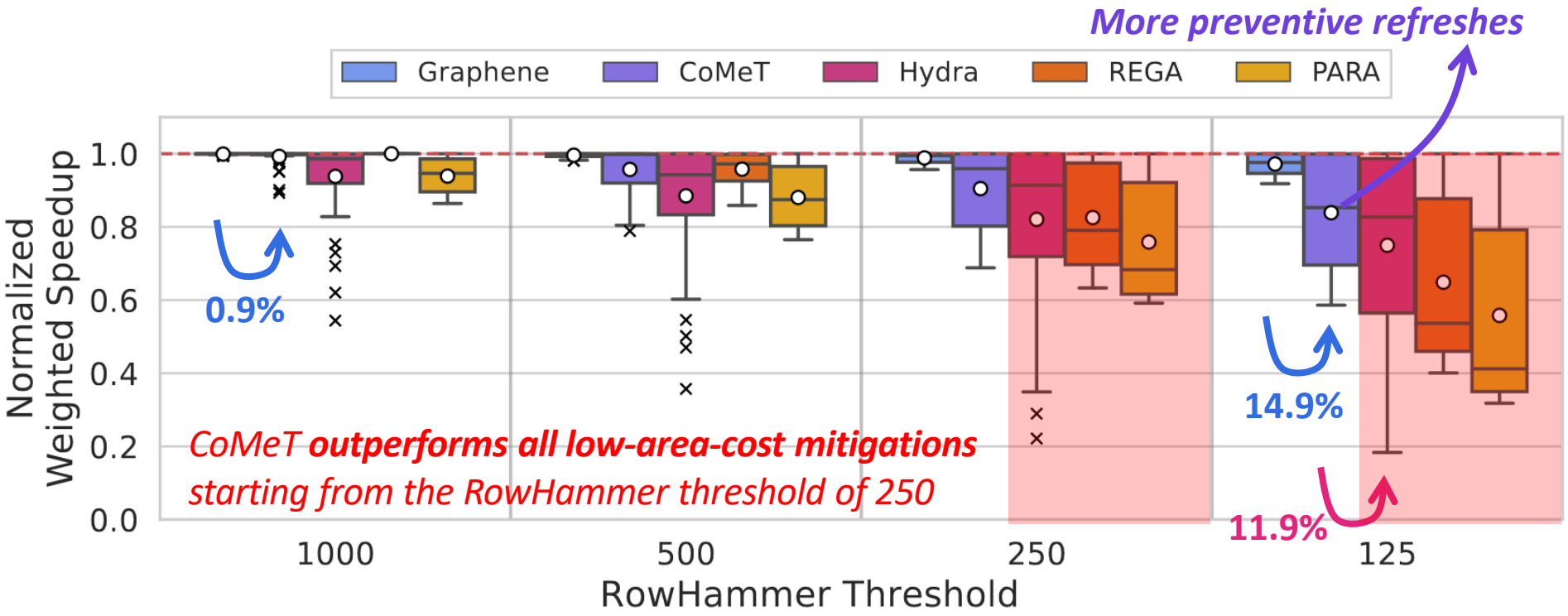


CoMeT's performance overhead over Graphene is **0.9%** and **14.9%** at N_{RH} = 1K and 125, respectively

CoMeT outperforms Hydra for all RowHammer thresholds (by up to **3.2×** and **11.9%** on average at N_{RH} = 125)

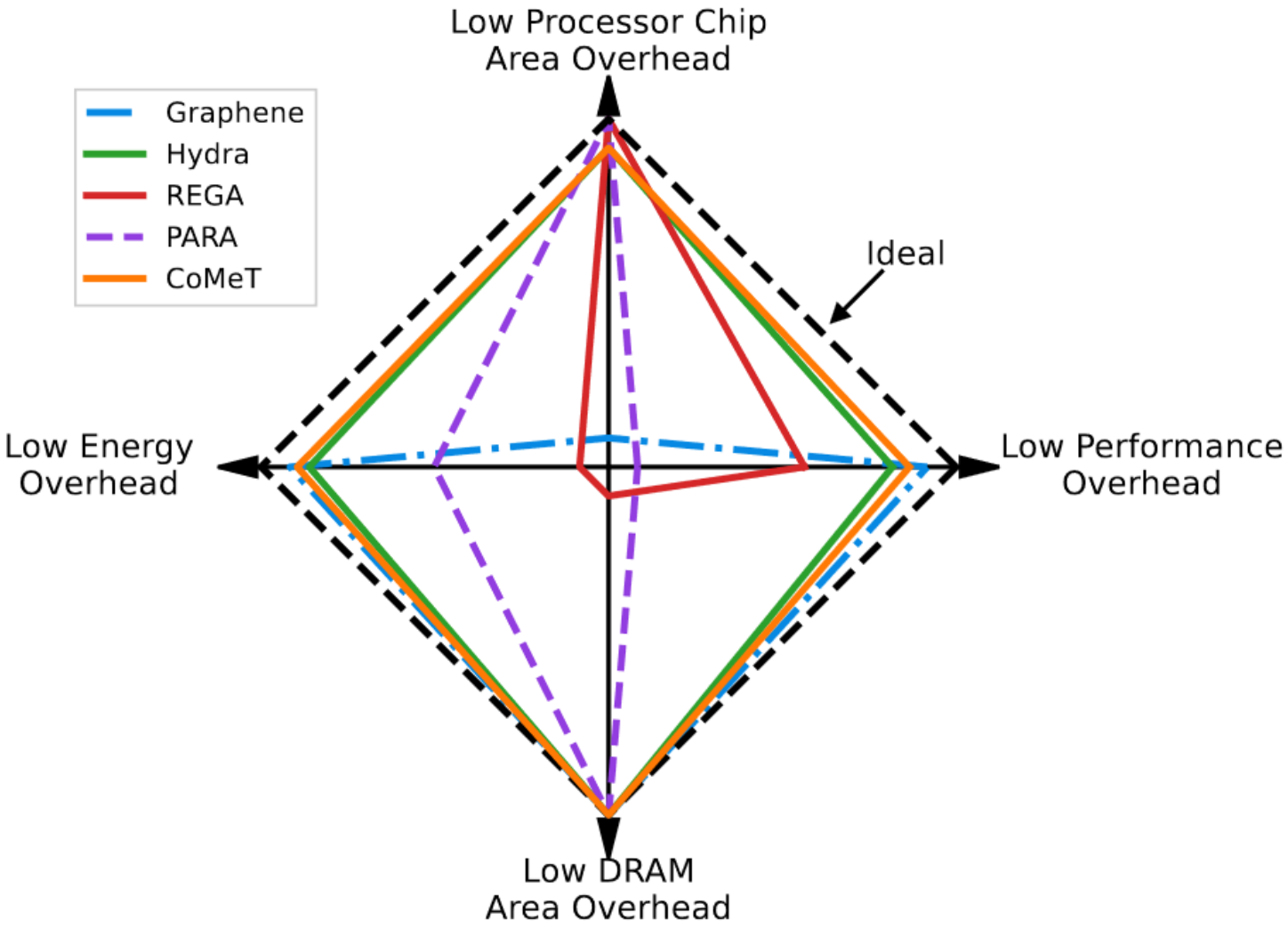
CoMeT outperforms *all* low-cost RowHammer mitigations starting from N_{RH}=250

Performance Comparison:
8-Core Workloads

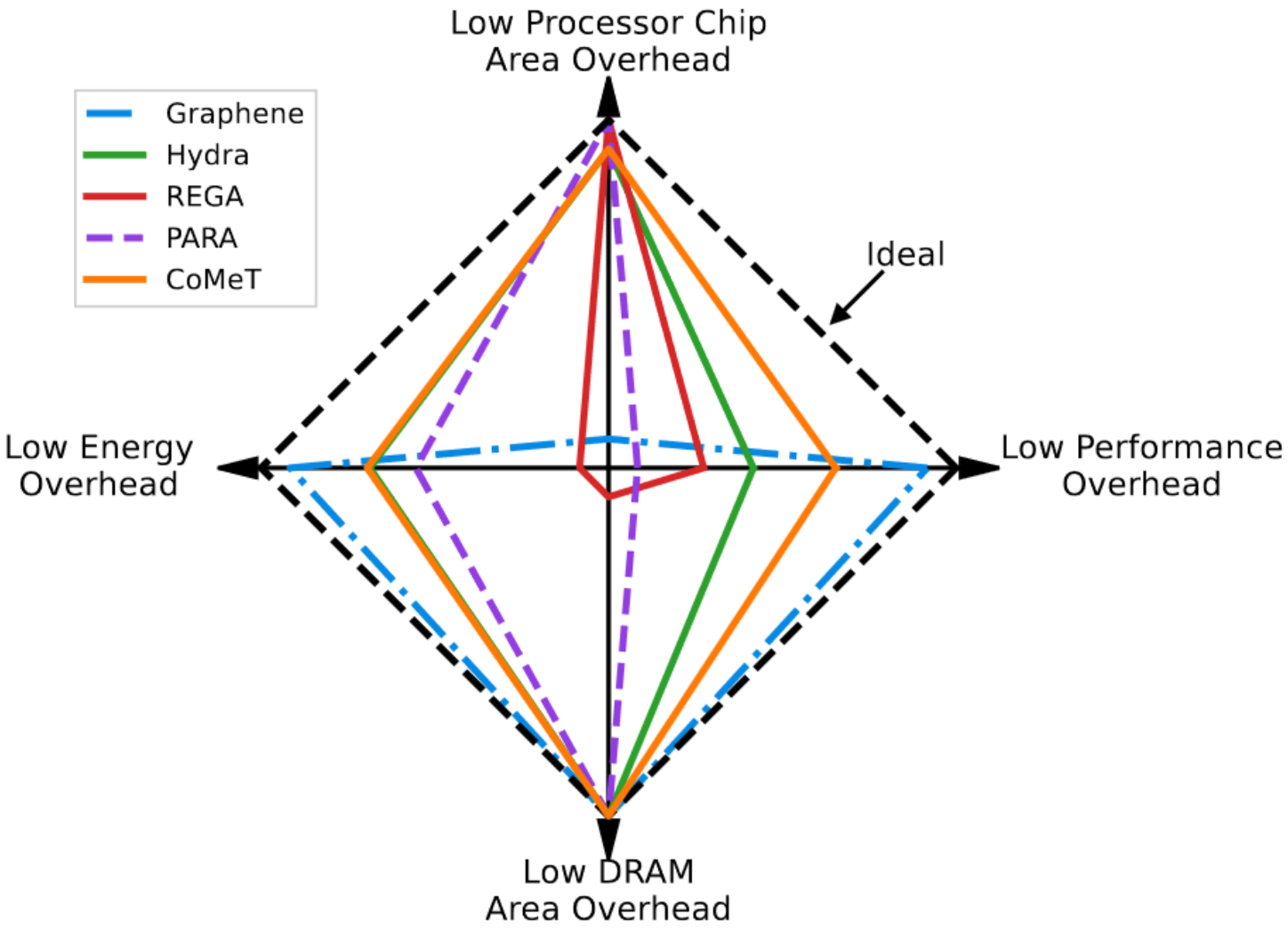


CoMeT maintains a performance overhead between Graphene's and Hydra's performance overheads

Single-Core Comparison – Radar Chart



Multi-Core Comparison – Radar Chart



Evaluated Workloads and Their Characteristics

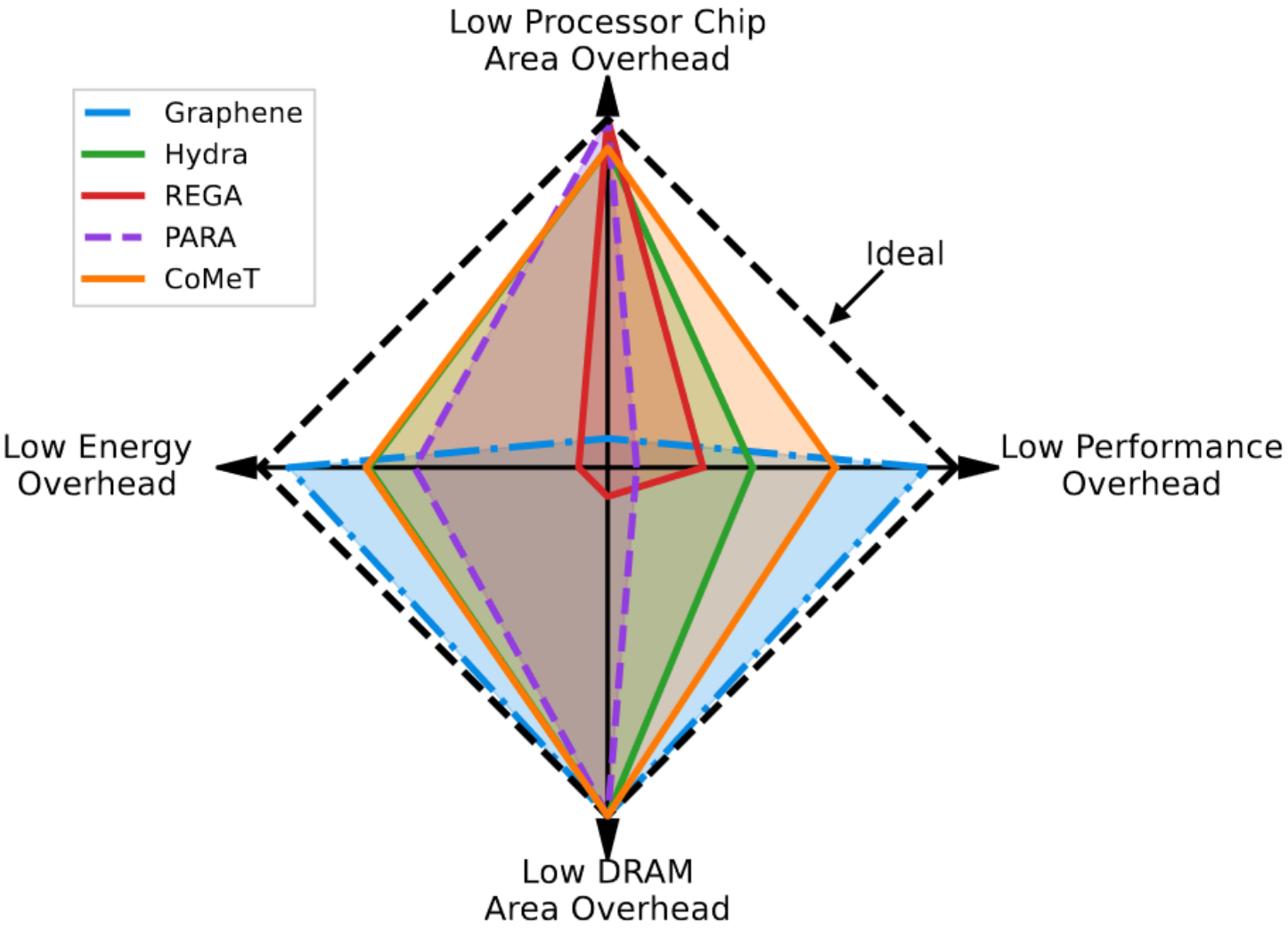
Table 3: Evaluated Workloads and Their Characteristics

RBMPKI	Workloads
[10+) (High)	519.lbm (5049 MB/s), 459.GemsFDTD (4788 MB/s), 450.soplex (3212 MB/s), h264_decode (11284 MB/s), 520.omnetpp (2567 MB/s), 433.milc (3595 MB/s), 434.zeusmp (5115 MB/s), bfs_dblp (12135 MB/s), 429.mcf (5588 MB/s), 549.fotonik3d (4428 MB/s), 470.lbm (6489 MB/s), bfs_ny (12146 MB/s), bfs_cm2003 (12138 MB/s), 437.leslie3d (3806 MB/s)
[2, 10) (Medium)	510.parest (92 MB/s), 462.libquantum (6089 MB/s), tpch2 (3612 MB/s), wc_8443 (1772 MB/s), ycsb_aserver (1080 MB/s), 473.astar (2473 MB/s), jp2_decode (1390 MB/s), 436.cactusADM (1915 MB/s), 557.xz (1113 MB/s), ycsb_cserver (842 MB/s), ycsb_eserver (721 MB/s), 471.omnetpp (96 MB/s), 483.xalancbmk (187 MB/s), 505.mcf (3760 MB/s), wc_map0 (1768 MB/s), jp2_encode (1706 MB/s), tpch17 (2553 MB/s), ycsb_bserver (854 MB/s), tpcc64 (1472 MB/s), 482.sphinx3 (968 MB/s)
[0, 2) (Low)	502.gcc (180 MB/s), 544.nab (78 MB/s), h264_encode (0.10 MB/s), 507.cactuBSSN (1325 MB/s), 525.x264 (109 MB/s), ycsb_dserver (659 MB/s), 531.deepsjeng (105 MB/s), 526.blender (56 MB/s), 435.gromacs (259 MB/s), 523.xalancbmk (180 MB/s), 447.dealII (24 MB/s), 508.namd (104 MB/s), 538.imagick (8 MB/s), 445.gobmk (97 MB/s), 444.namd (104 MB/s), 464.h264ref (17 MB/s), ycsb_abgsave (362 MB/s), 458.sjeng (131 MB/s), 541.leela (4 MB/s), tpch6 (675 MB/s), 511.povray (1 MB/s), 456.hmmer (28 MB/s), 481.wrf (7 MB/s), grep_map0 (381 MB/s), 500.perlbench (642 MB/s), 403.gcc (79 MB/s), 401.bzip2 (59 MB/s)

CoMeT's Performance at High RowHammer Thresholds

We evaluate the performance of 61 single-core applications at high RowHammer thresholds of 2000 and 4000. We observe that CoMeT incurs 0.015% and 0.0053% average performance overheads, at $N_{RH} = 2000$ and $N_{RH} = 4000$, respectively. We conclude that CoMeT has negligible performance overhead at high RowHammer thresholds.

Summary of the Results



Storage Overhead of Graphene

Table 1: Storage overhead of a performance-optimized state-of-the-art RowHammer mitigation [86].

N_{RH}	1000	500	250	125
Storage (KB)	207.19	498.44	765.00	1466.25

Performance Overhead of Hydra

